

计算机体系结构

L05 指令并行高级技术

计算机体系结构

背景知识 (自学) ③

存储层次 ④

单处理器 (指令级并行) ⑤

简单流水线 ④

复杂流水线 ②

分支预测 ③

指令并行高级技术

多发射

超标量 (Superscalar)

超长指令字 (VLIW)

多线程 (Multithreading)

细粒度多线程

粗粒度多线程

同步多线程 (SMT)

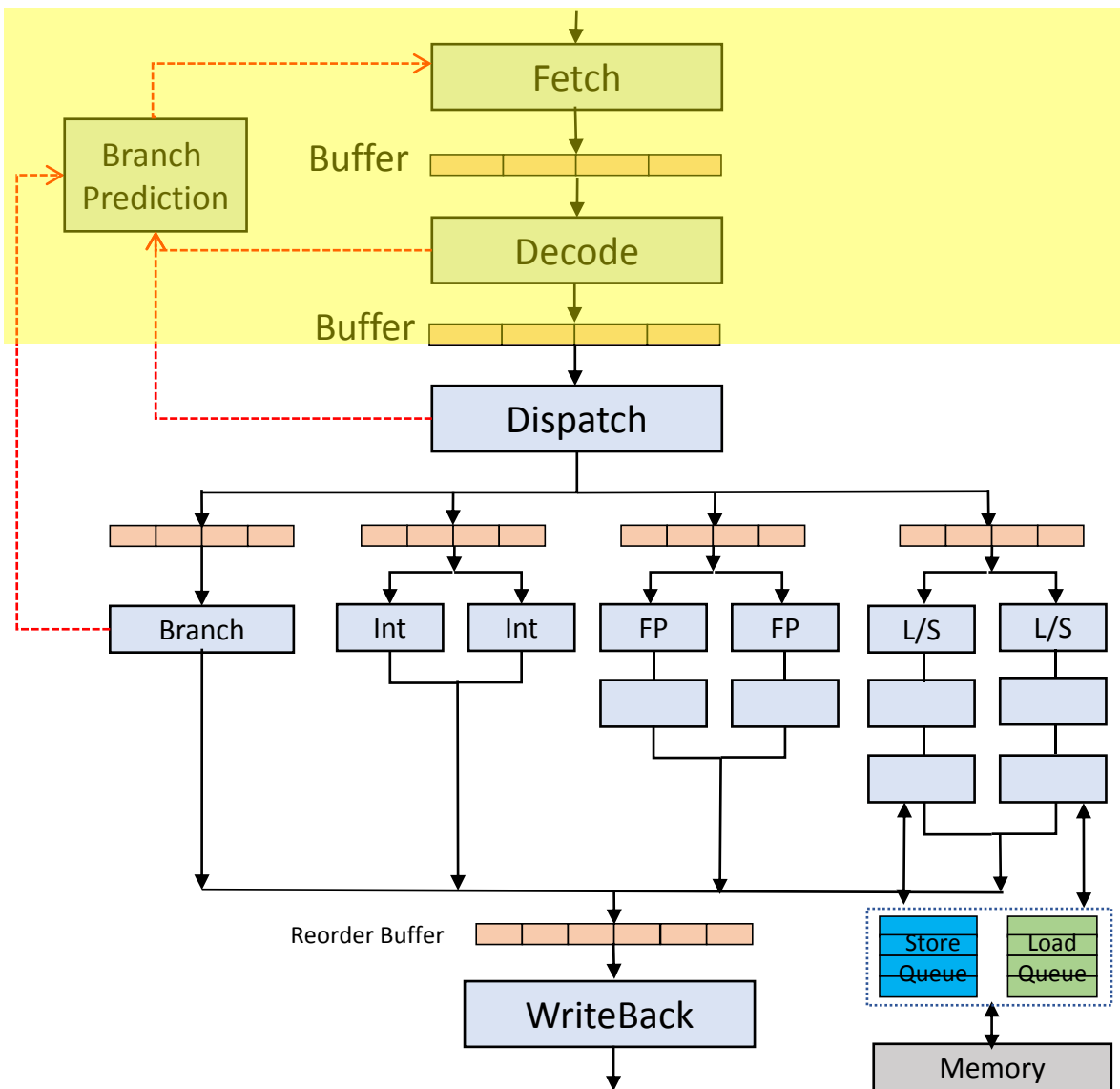
多处理器 (线程级并行) ⑤

加速器 (数据并行) ③

多发射

(Multi-Issue)

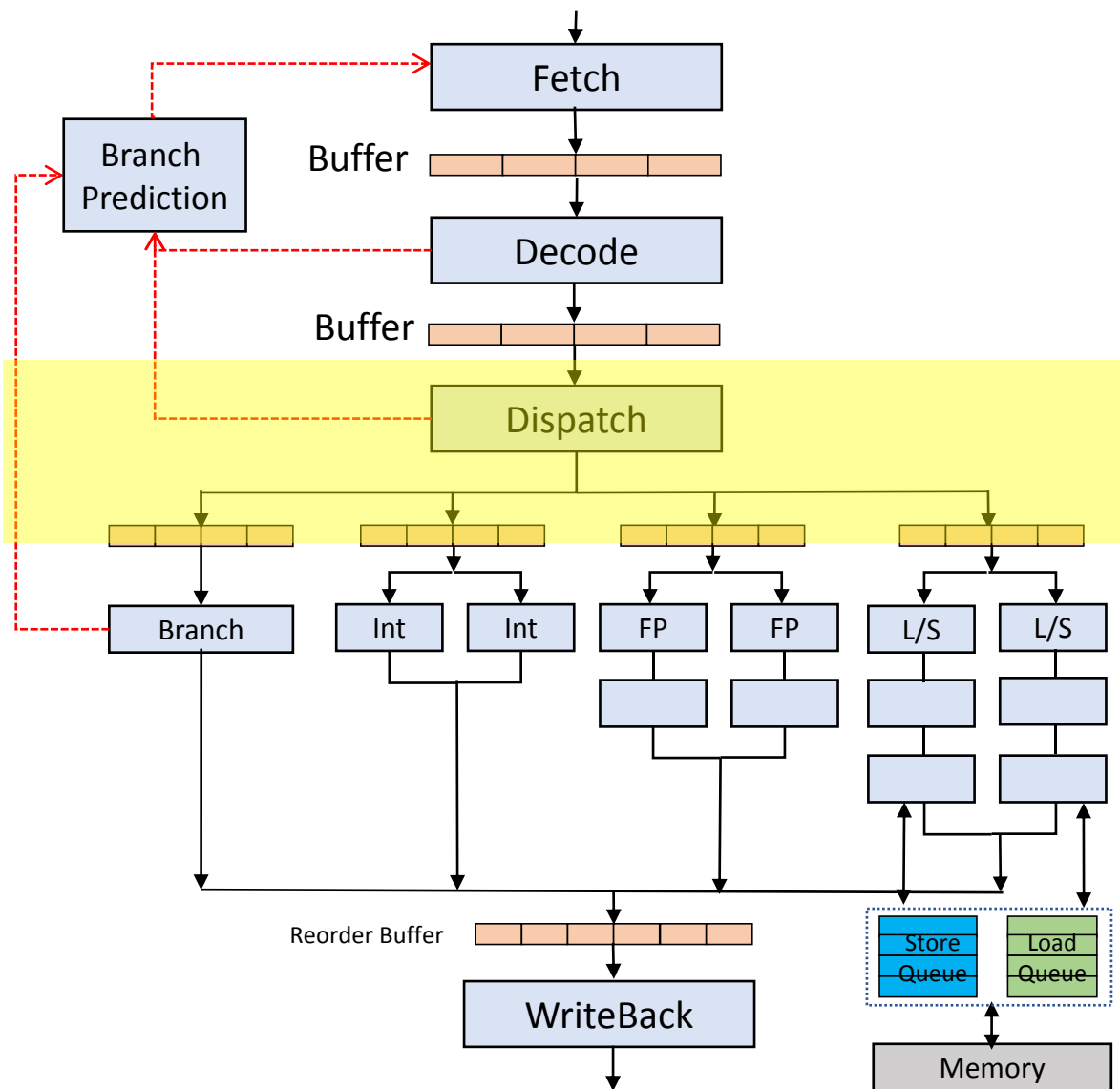
单发射、乱序流水线



1. 取指 & 译码

取指&译码，每个周期1条

单发射、乱序流水线



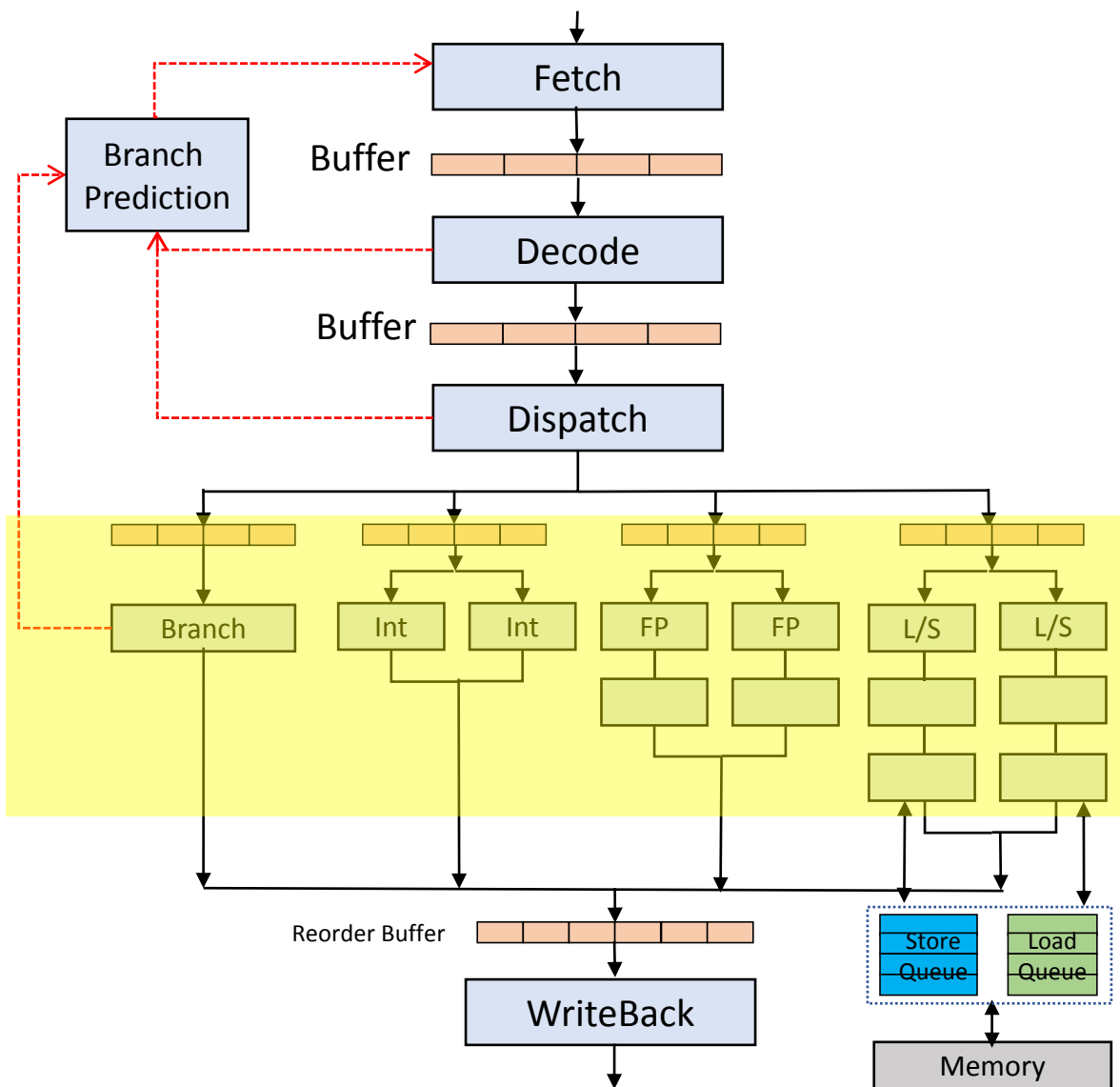
1. 取指 & 译码

取指&译码，每个周期1条

2. 指令发送

将指令送入保留站，解析数据相关性(重命名)

单发射、乱序流水线



1. 取指 & 译码

取指&译码，每个周期1条

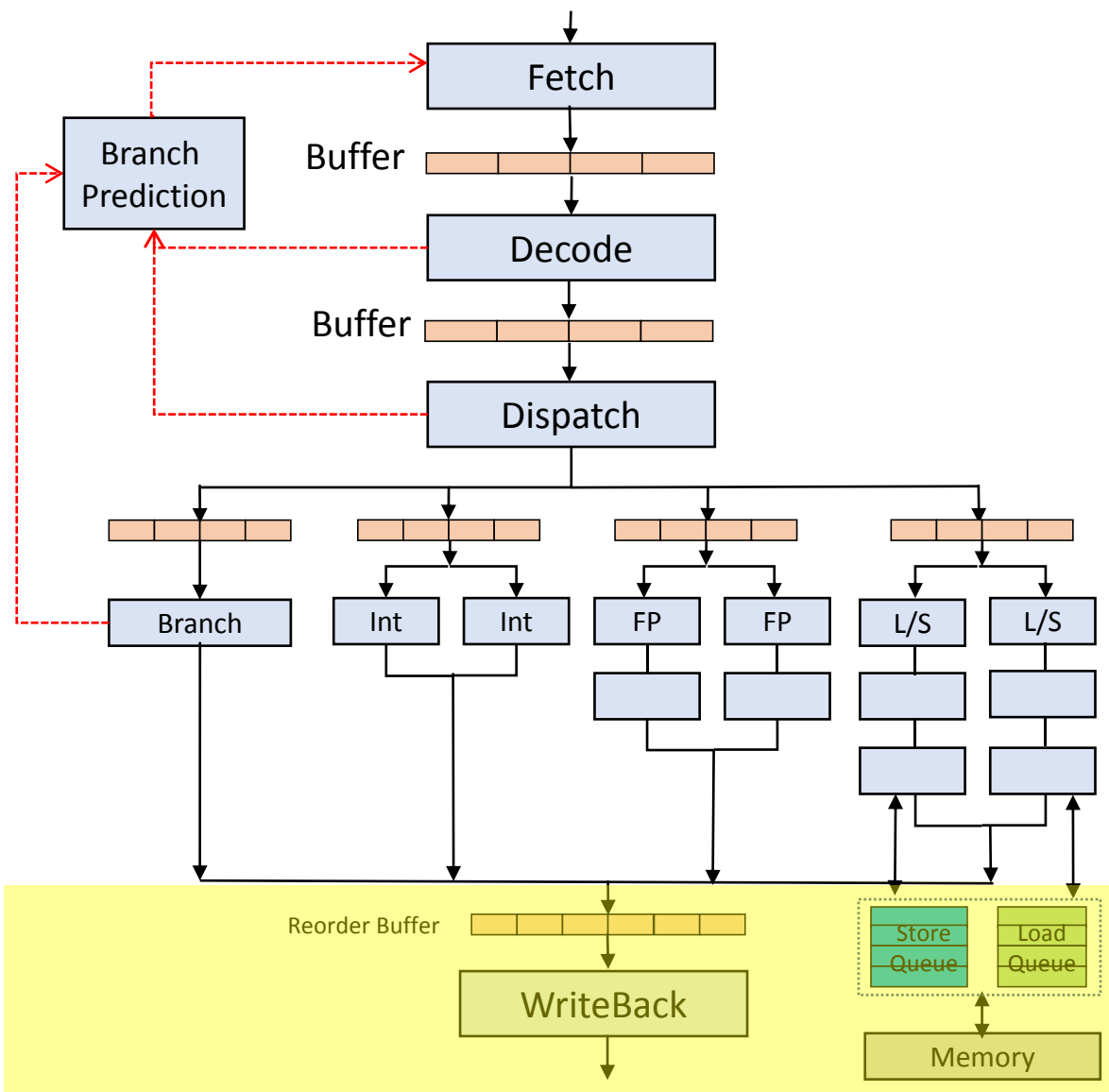
2. 指令发送

将指令送入保留站，解析数据相关性(重命名)

3. 指令发射

将准备好的指令送入执行单元

单发射、乱序流水线



1. 取指 & 译码

取指&译码，每个周期1条

2. 指令发送

将指令送入保留站，解析数据相关性(重命名)

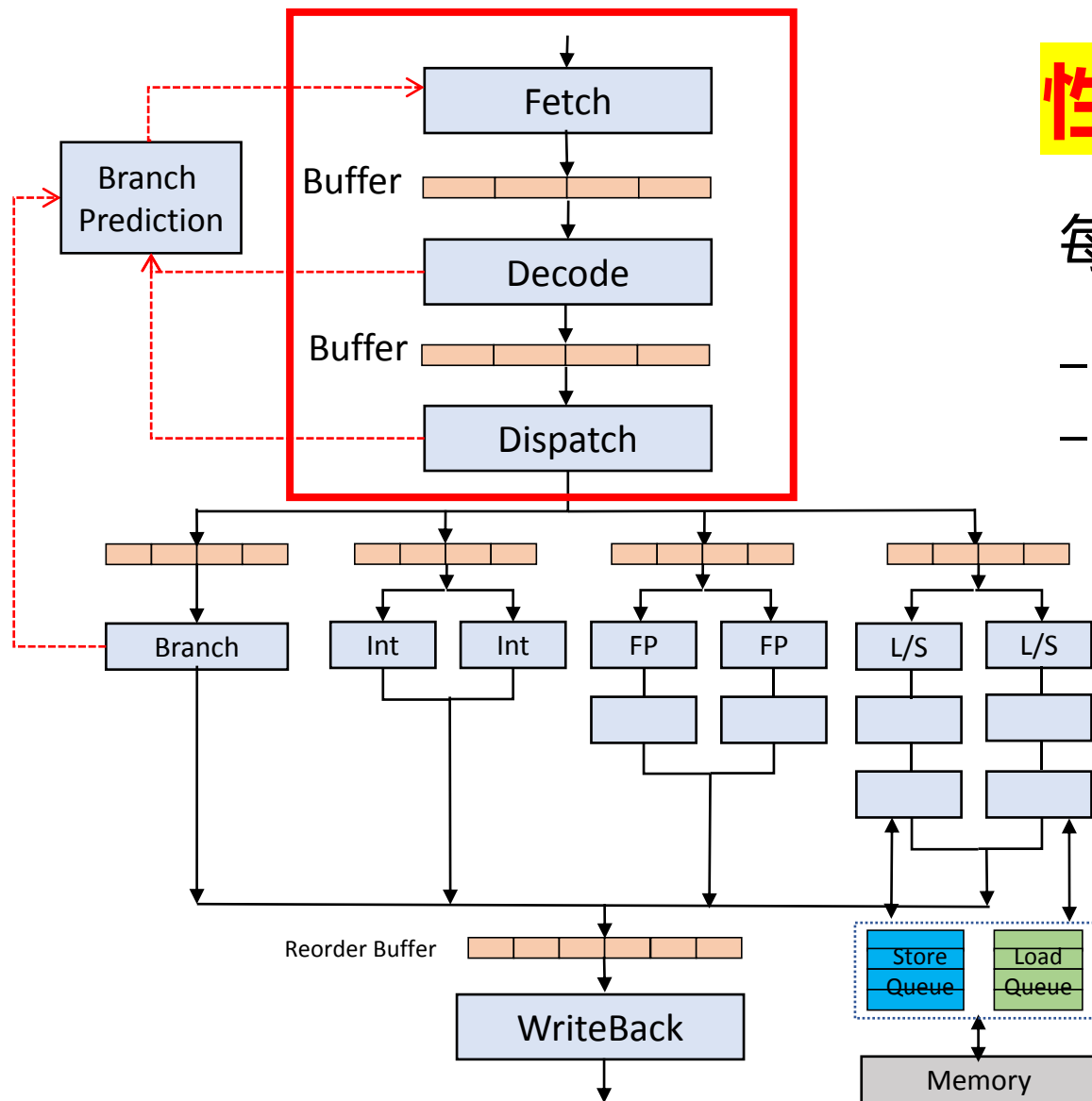
3. 指令发射

将准备好的指令送入执行单元

4. 指令提交

提交指令，更新系统状态

单发射、乱序流水线



性能瓶颈！

每个周期最多取一条指令

- 性能上限 ($CPI = IPC = 1$)
- 实际性能达不到性能上限

能否同时发射多条指令？

□ 可行:

ADD r1, r2 -> r3

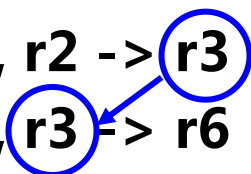
ADD r4, r5 -> r6

✓ 没有相关性!

□ 不行:

SUB r1, r2 -> r3

SUB r4, r3 -> r6



✓ 有相关性

□ 同时发射3条、或者更多条指令呢？

多发射方案

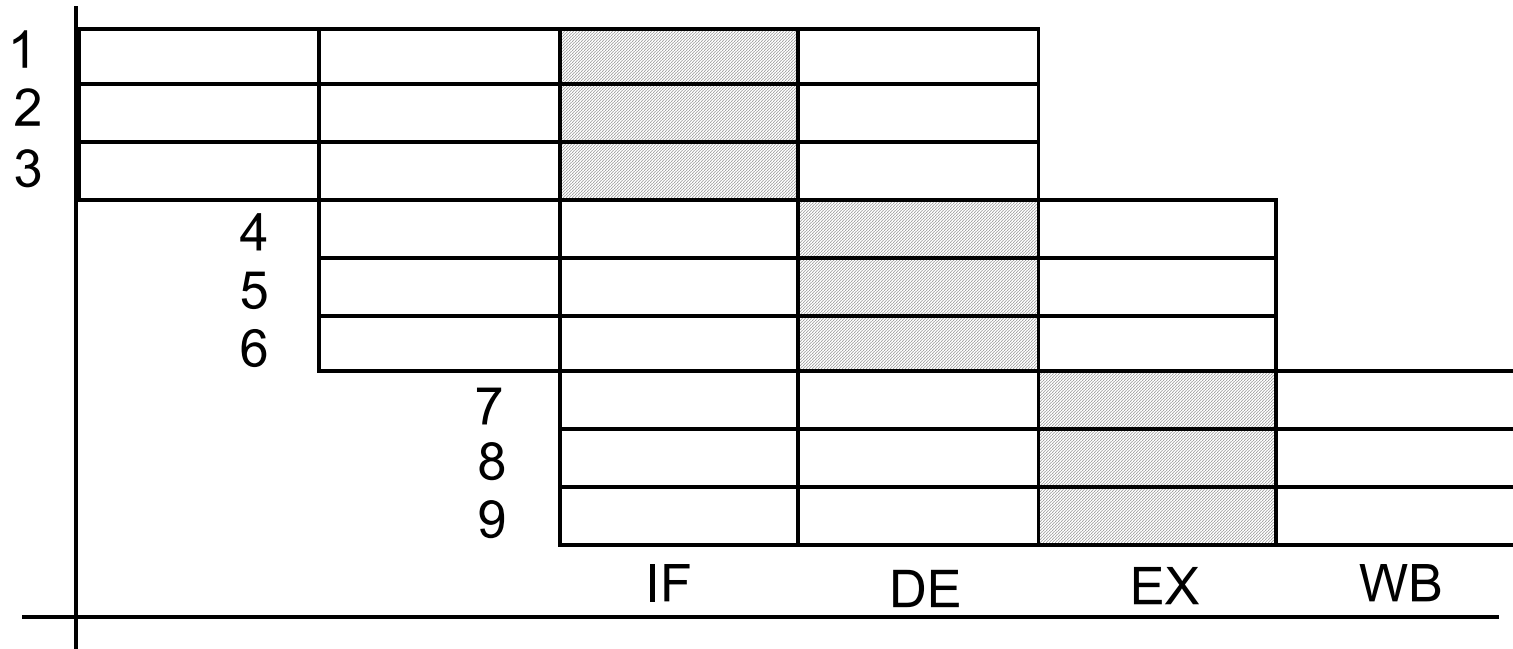
- ❑ **Superscalar(超标量):** 每周期发射多条指令(个数不固定, 1 to 8), 由编译器或者硬件调度
 - IBM PowerPC, Sun UltraSparc, DEC Alpha, Pentium 4, i7
- ❑ **Very Long Instruction Words (VLIW, 超长指令字):** 将多条指令(4-16)打包成一条固定格式的长指令, 由编译器静态完成
 - Intel architecture-64 (IA-64) 64-bit address (renamed: “Explicitly Parallel Instruction Computer (EPIC)”)

超标量
(Superscalar)

Superscalar

□ 每个周期发射N条指令

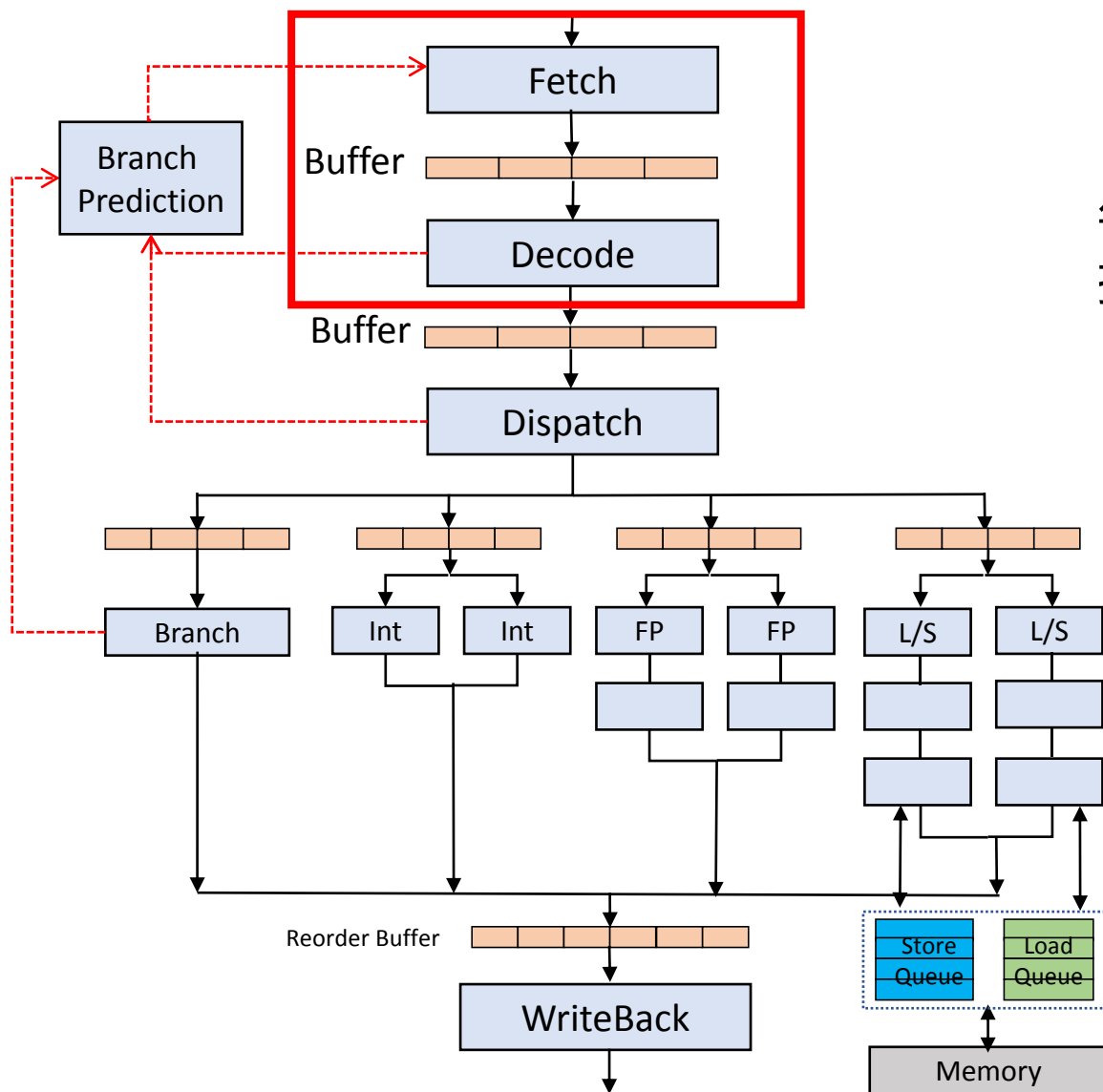
□ 最高IPC = N



Superscalar设计要点 - 取指

取指&译码

每个周期取N条指令，N条指令同时译码

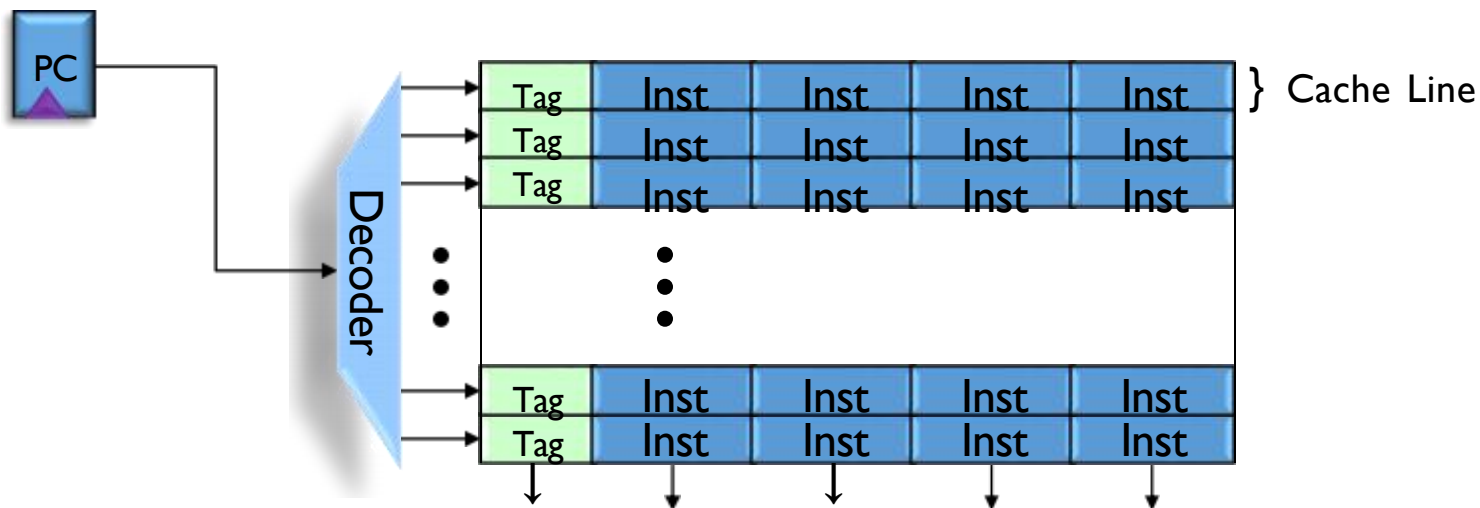


指令Cache设计

□ Cache line大小正好容纳N条指令

✓ 假设Superscalar宽度为N

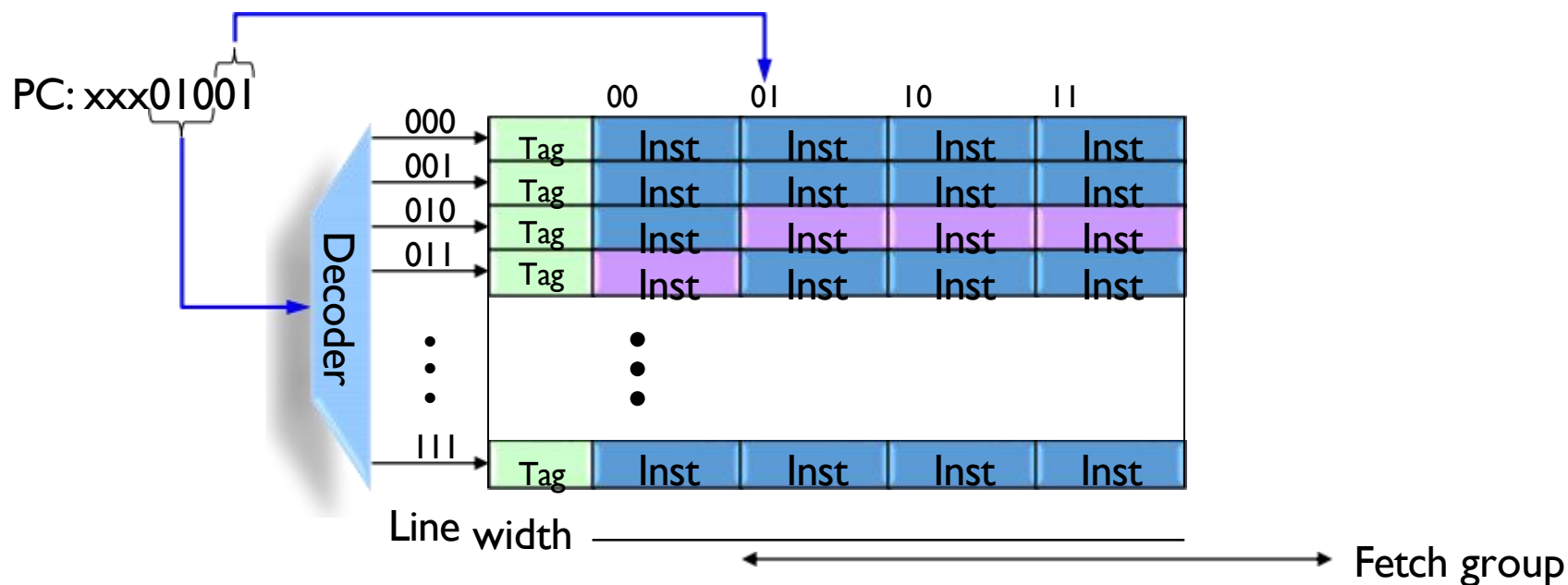
□ 如果第1条指令的PC正好是cache line的开头，直接取整个cache line (4条指令)，单周期可完成



Cache line 未对齐

□ 第1条指令地址不在cache line的起始（分支跳转导致）

✓ 同一组指令跨越两个cache line



Cache line 未对齐

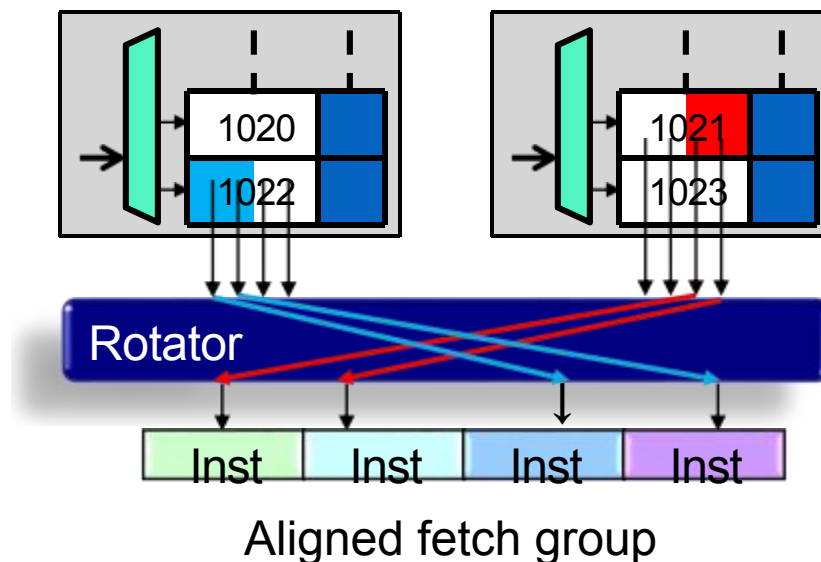
□ 解决方案1：分别读取两个cache line

✓ 需要两个周期

□ 解决方案2：同时取两个cache line

✓ 设计更复杂，可能增加延迟

□ 其他技术

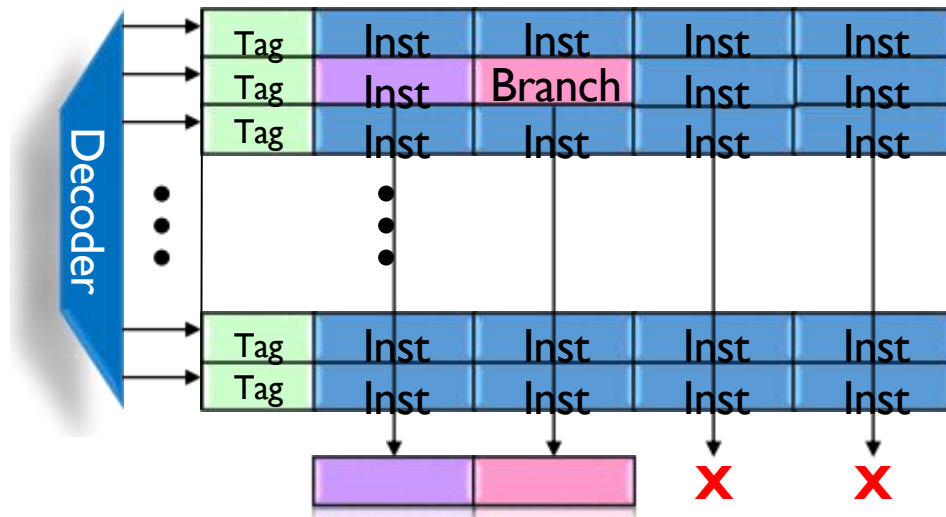


分支语句

□ 可能导致指令流不连续

□ 可能导致cache line未对齐

✓ 跳转地址不在cache line开头



Superscalar中的分支预测

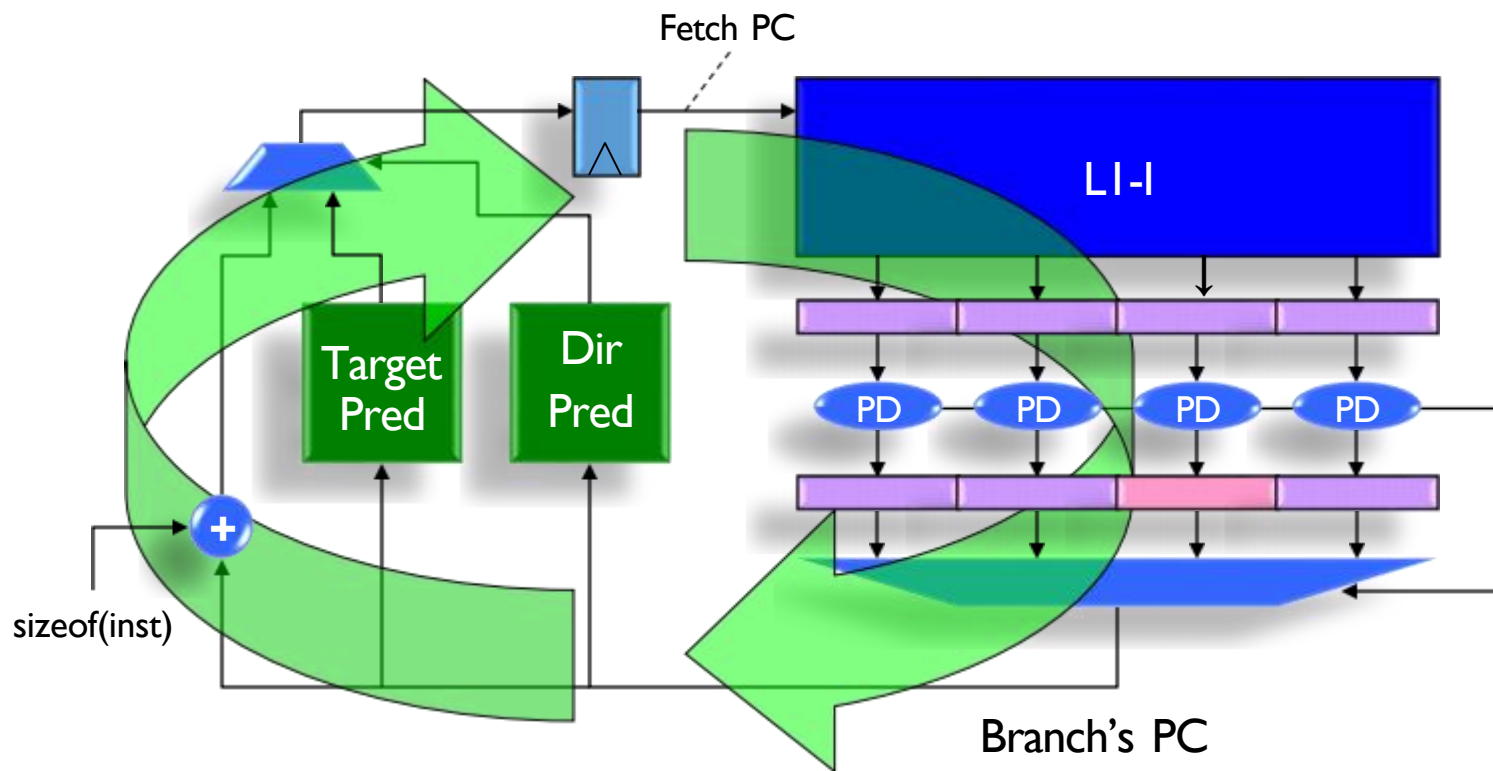
□ 同一组指令(N条)可能包含多条分支语

□ 对哪几条分支语句进行预测?

- ✓ **简单方案:** 只预测第1条指令
- ✓ **困难:** 预测最先发生跳转的那条
- ✓ **更困难:** 同时预测多条分支语句

Superscalar中的分支预测

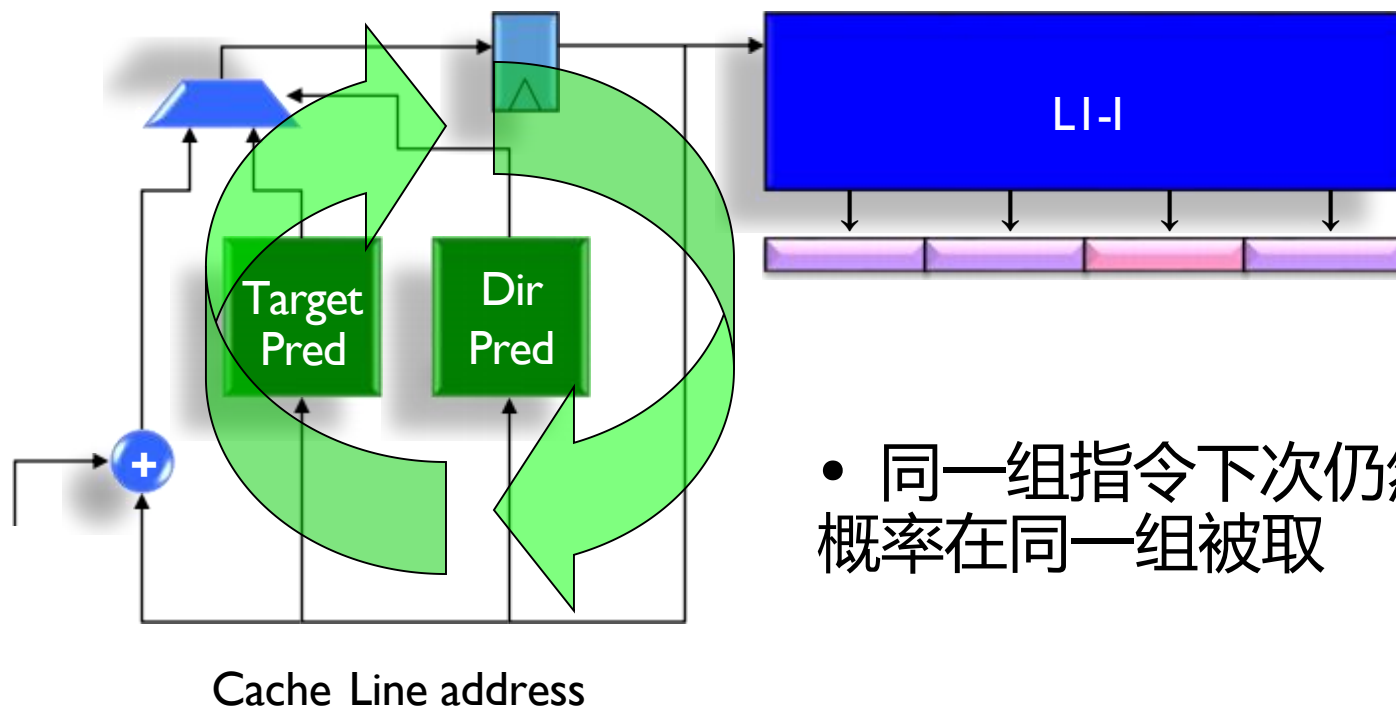
方案一：对指令进行部分译码



通过部分译码分析哪些是分支语句，再进行预测

Superscalar中的分支预测

□ 方案二：根据同一组指令的首地址进行预测



- 同一组指令下次仍然很大概率在同一组被取

预测该组指令是否会跳转、跳转到哪里

指令译码

□ 需要同时为多条指令进行译码

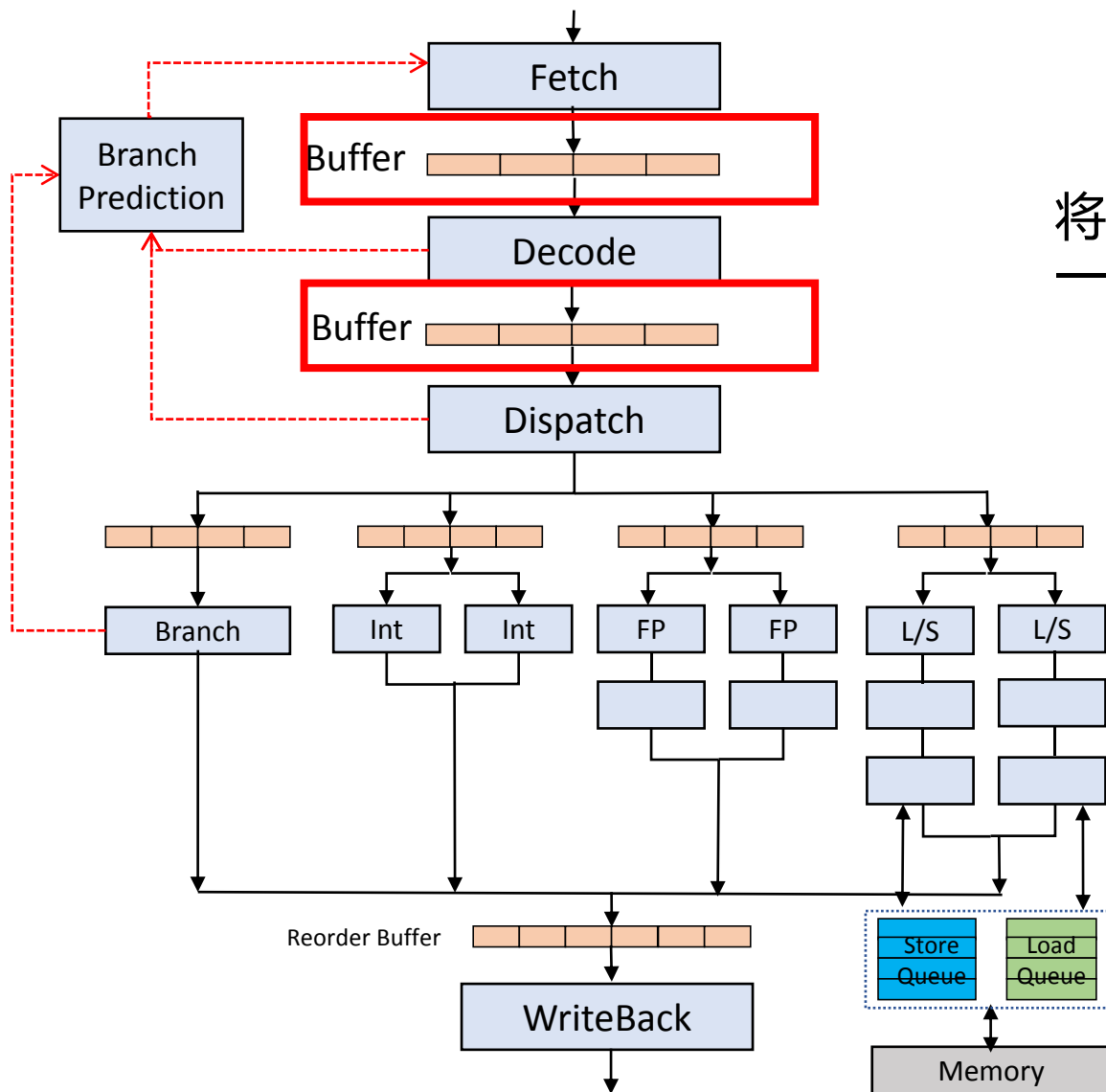
- 增加并行译码电路

□ 单条指令译码与单发射流水线无本质区别

Superscalar 设计要点 - 指令缓存

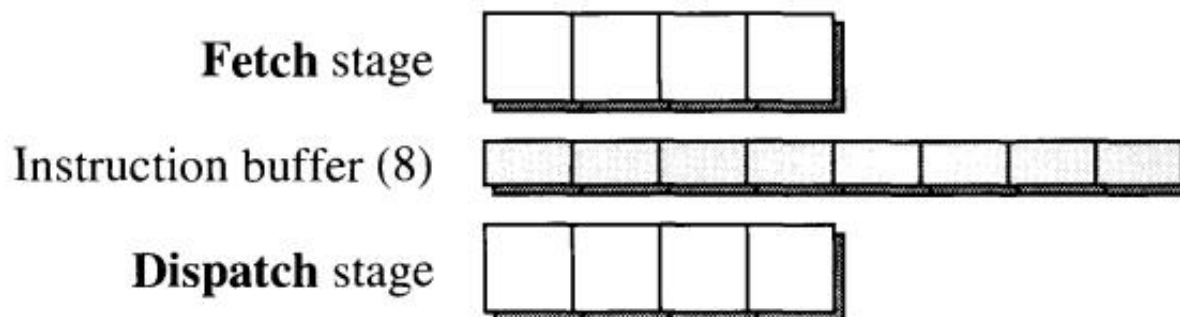
指令缓存

将各个阶段进行解耦，防止某一阶段阻塞、后面阶段都阻塞



Superscalar 设计要点 - 指令缓存

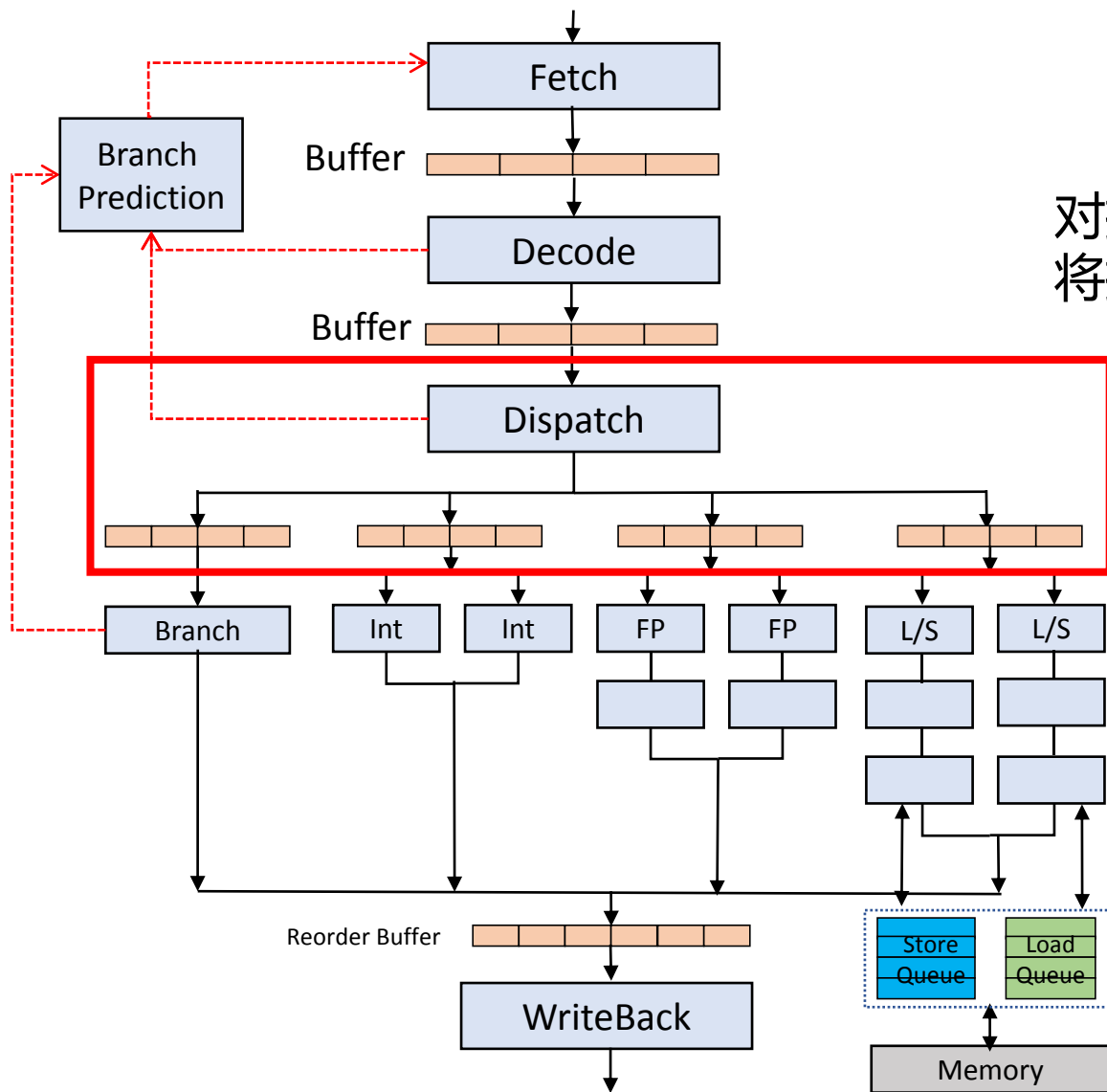
- ❑ 指令缓存位于fetch阶段和dispatch阶段之间
- ❑ 如果fetch跟不上，或者dispatch阻塞，指令缓存提供缓冲
- ❑ 译码结束之后的指令先放入指令缓存，dispatch模块从指令缓存取指令进行发送
- ❑ 指令缓存要能够支持单个周期存/取多条指令



Superscalar 设计要点 - 指令发送

指令发送

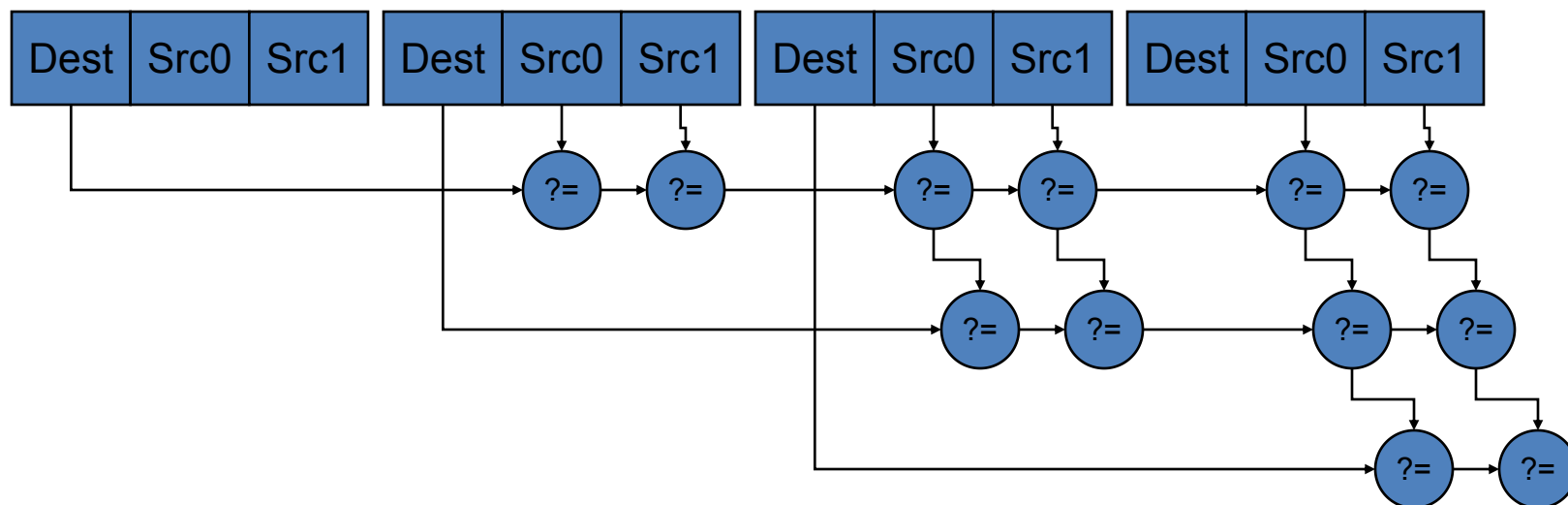
对指令之间的相关性进行解析，
将指令送入相应的保留站



Superscalar 设计要点 - 指令发送

- ❑ 检查指令缓存中的指令，看是否满足发送到保留站的条件 (RS空位，ROB空位)
- ❑ 如果可以发送，为指令分配保留站entry, ROB entry，发送指令到保留站
 - ✓ 完成了相关性解析和寄存器重命名
- ❑ 逻辑上，指令**必须按照顺序发送**
 - ✓ 实现上，**采用逻辑电路并行完成**
- ❑ 通常，**每个周期最多发送N条指令**，每个功能单元最多接收1条指令

Superscalar 设计要点 - 指令发送



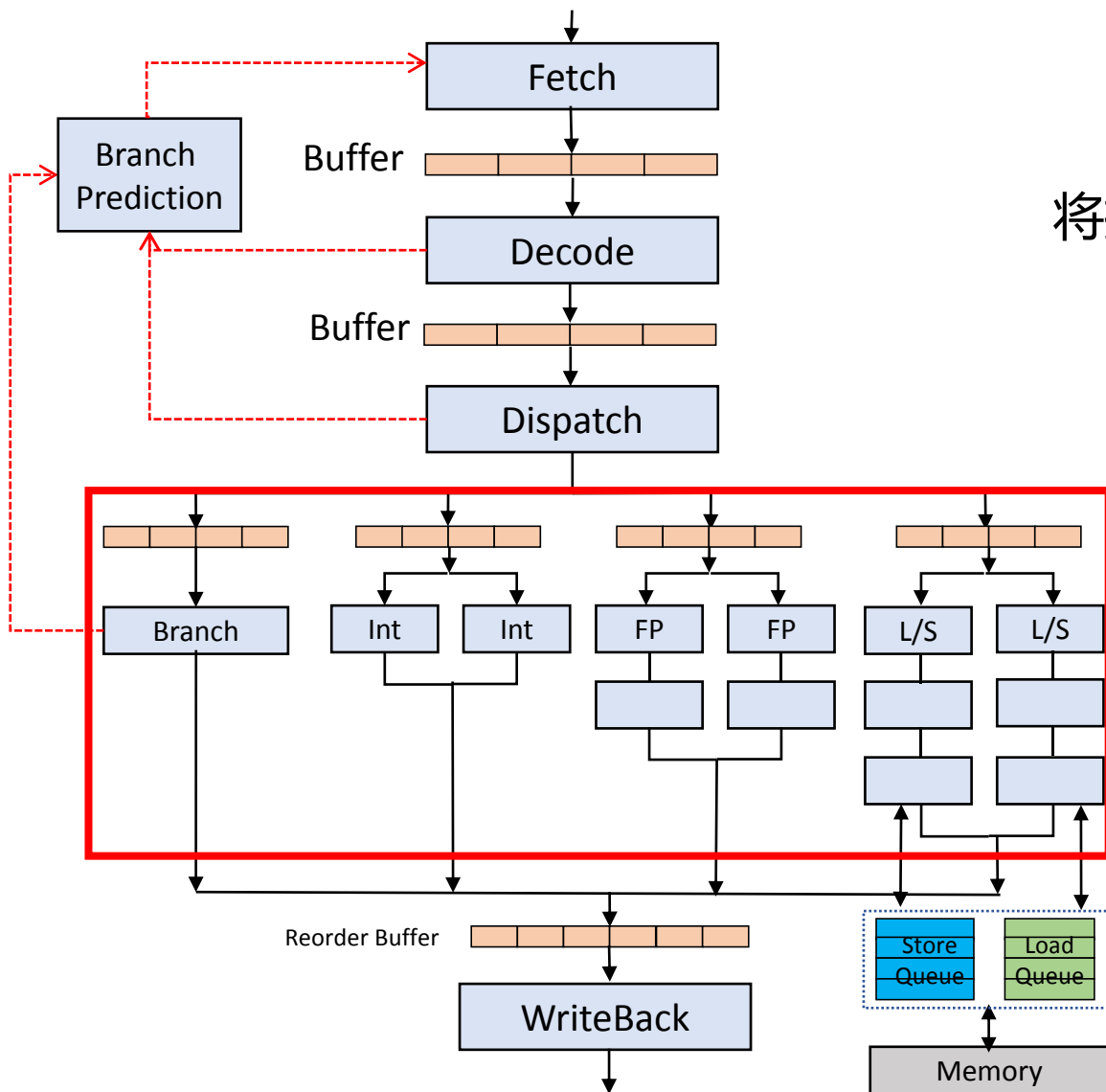
□ 逻辑电路检查指令之间的相关性

- ✓ 如果有相关性，可分批发送 (保证同一批发送的指令不关)

Superscalar 设计要点 - 指令发射

指令发射

将指令从保留站发送到执行单元



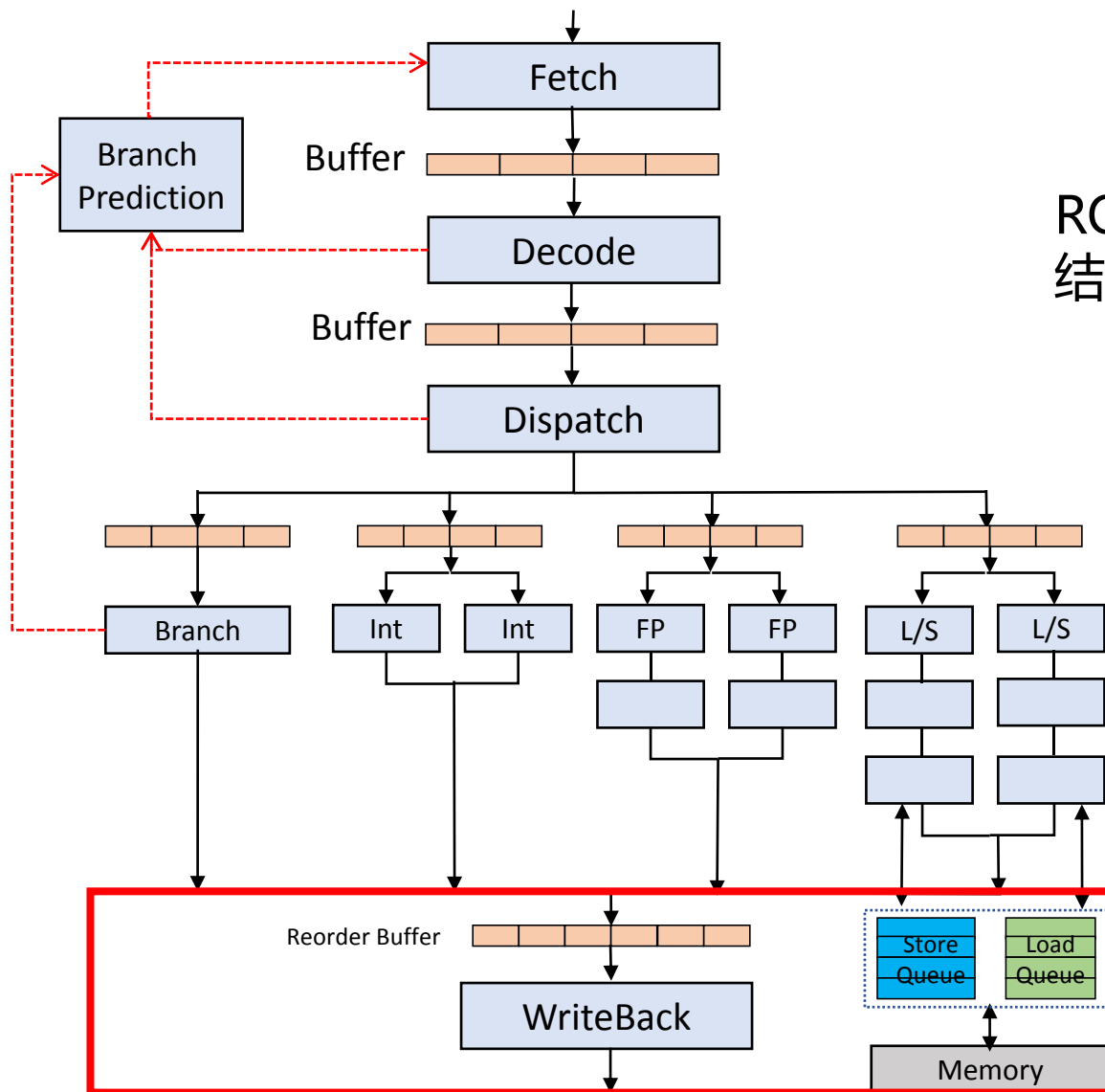
Superscalar 设计要点 - 指令发射

- 将准备好的指令从保留站发送到执行单元
- 每个周期可**发射多条指令**
 - ✓ 同一个执行单元一般只最多只发射一条
- 可能有多条指令满足发射条件，需要**选择逻辑**确定最终发射哪些指令
 - ✓ 还需考虑结构冲突等因素

Superscalar 设计要点 - 指令提交

指令提交

ROB中的指令按照顺序进行提交结果，更新系统状态



Superscalar 设计要点 - 指令提交

- 指令执行完后，先将结果放入ROB
- ROB中已经完成的指令按顺序提交，**每个周期可以提交多条指令**
 - ✓ 采用逻辑电路实现

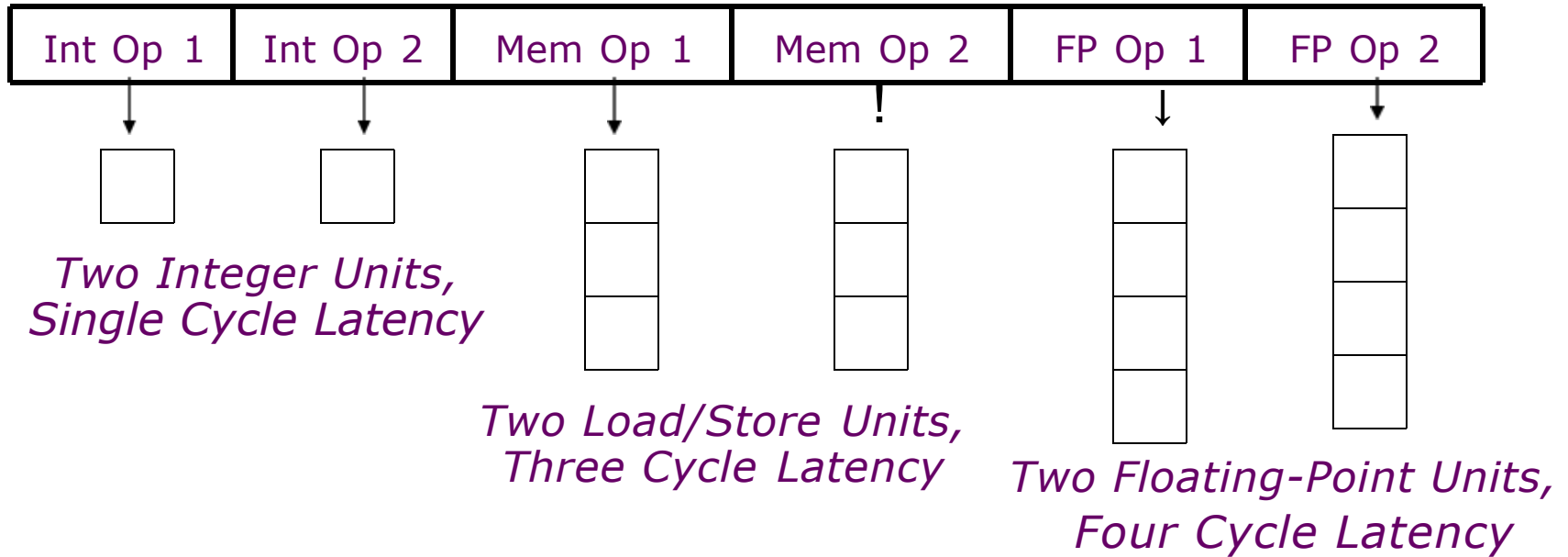
Superscalar 设计要点总结

- **取指**：一个周期同时取多条指令
 - ✓ Cache line设计，解决未对齐问题
 - ✓ 并行decode
- **指令发送**：多条指令同时发送
 - ✓ 并行完成相关性检测和寄存器重命名
- **指令发射**：多条指令同时发射
 - ✓ 检查资源冲突等
- **指令执行**：多个功能单元并行执行
 - ✓ 和单发射无本质区别
- **指令提交**：并行提交多条指令
 - ✓ 按顺序并行提交多条指令

超长指令字

(Very Long Instruction Word)

超长指令字 (VLIW)



- 多个操作打包到一条指令
- 指令格式是固定的（哪个位置放哪种操作）
- 每种操作的延迟是指定的

□ 依赖编译器进行调度

- 硬件设计简单
- 时钟频率可能较高(因为硬件简单)
- 需要额外的内存和带宽(代码长度增加)

□ 编译器负责解决各种依赖关系

- 编译器需要知道处理器和内存细节, 例如执行单元的个数和类型, 处理延迟, 内存延迟等.

□ 不同型号的处理器编译器不能共享

循环语句

```
for (i=0; i<N; i++)  
    B[i] = A[i] + C;
```

Compile

```
loop: ld f1, 0(r1)  
      add r1, 8  
      fadd f2, f0, f1  
      sd f2, 0(r2)  
      add r2, 8  
      bne r1, r3, loop
```

loop:

Schedule

Int1	Int 2	M1	M2	FP+	FPx
add r1		ld			
				fadd	
add r2	bne	sd			

6个操作打包成3条指令，因为相关性
指令之间还需要stall

$$1 \text{ fadd} / 8 \text{ cycles} = 0.125$$

循环展开

```
for (i=0; i<N; i++)  
    B[i] = A[i] + C;
```

展开4次

```
for (i=0; i<N; i+=4)  
{  
    B[i]    = A[i] + C;  
    B[i+1]  = A[i+1] + C;  
    B[i+2]  = A[i+2] + C;  
    B[i+3]  = A[i+3] + C;  
}
```

如果N不是4的整数倍，需要特殊处理

循环展开

展开4次

```
loop: ld f1, 0(r1)
      ld f2, 8(r1)
      ld f3, 16(r1)
      ld f4, 24(r1)
      add r1, 32
      fadd f5, f0, f1
      fadd f6, f0, f2
      fadd f7, f0, f3
      fadd f8, f0, f4
      sd f5, 0(r2)
      sd f6, 8(r2)
      sd f7, 16(r2)
      sd f8, 24(r2)
      add r2, 32
      bne r1, r3, loop
```

Schedule →

loop:

Int1	Int 2	M1	M2	FP+	FPx
		ld f1			
		ld f2			
		ld f3			
add r1		ld f4		fadd f5	
				fadd f6	
				fadd f7	
				fadd f8	
		sd f5			
		sd f6			
		sd f7			
add r2	bne	sd f8			

性能变化？

4 fadds / 11 cycles = 0.36

软件流水

先循环展开，再指令调度

```
loop: ld f1, 0(r1)
      ld f2, 8(r1)
      ld f3, 16(r1)
      ld f4, 24(r1)
      add r1, 32
      fadd f5, f0, f1
      fadd f6, f0, f2
      fadd f7, f0, f3
      fadd f8, f0, f4
      sd f5, 0(r2)
      sd f6, 8(r2)
      sd f7, 16(r2)
      add r2, 32
      sd f8, -8(r2)
      bne r1, r3, loop
```

首次循环

循环体

最后一次循环

Int1 Int 2 M1 M2 FP+ FPx

		ld f1			
		ld f2			
		ld f3			
add r1		ld f4			
		ld f1		fadd f5	
		ld f2		fadd f6	
		ld f3		fadd f7	
add r1		ld f4		fadd f8	

loop:

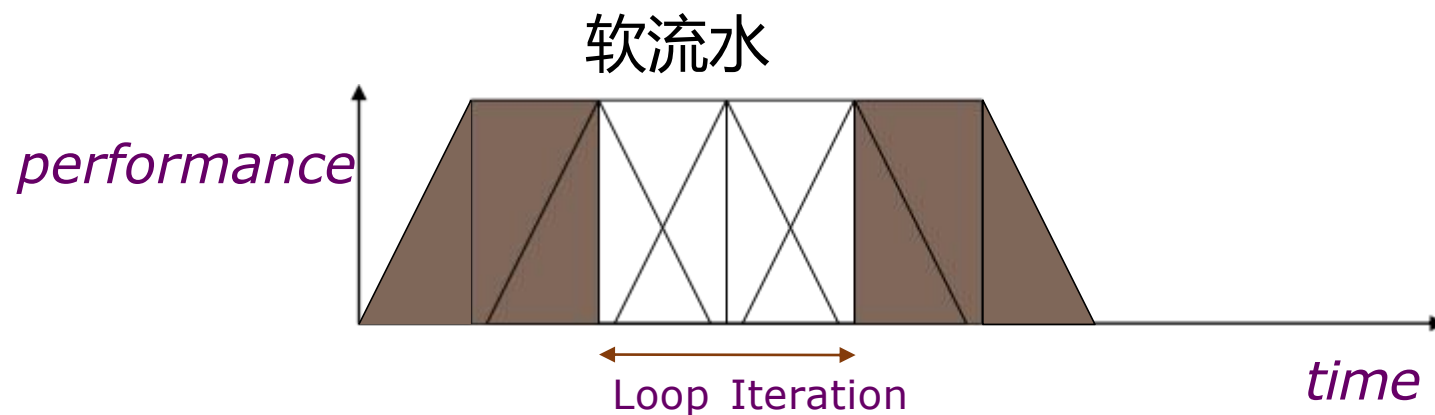
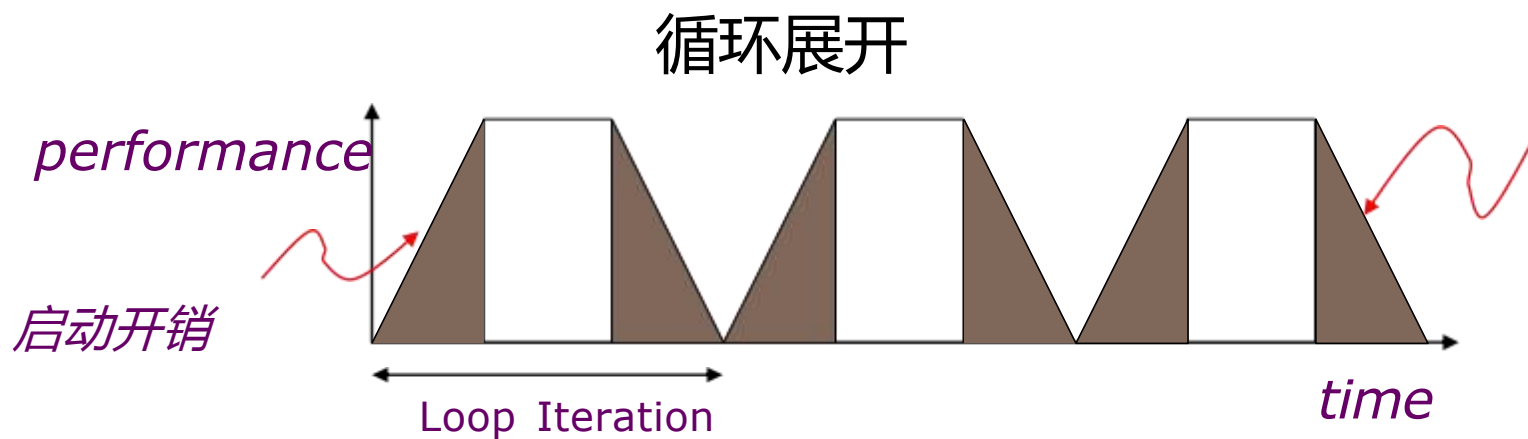
		ld f1	sd f5	fadd f5	
		ld f2	sd f6	fadd f6	
	add r2	ld f3	sd f7	fadd f7	
add r1 bne		ld f4	sd f8	fadd f8	

			sd f5	fadd f5	
	add r2		sd f6	fadd f6	
	bne		sd f7	fadd f7	
			sd f8	fadd f8	
			sd f5		

性能如何？

4 fadds / 4 cycles = 1

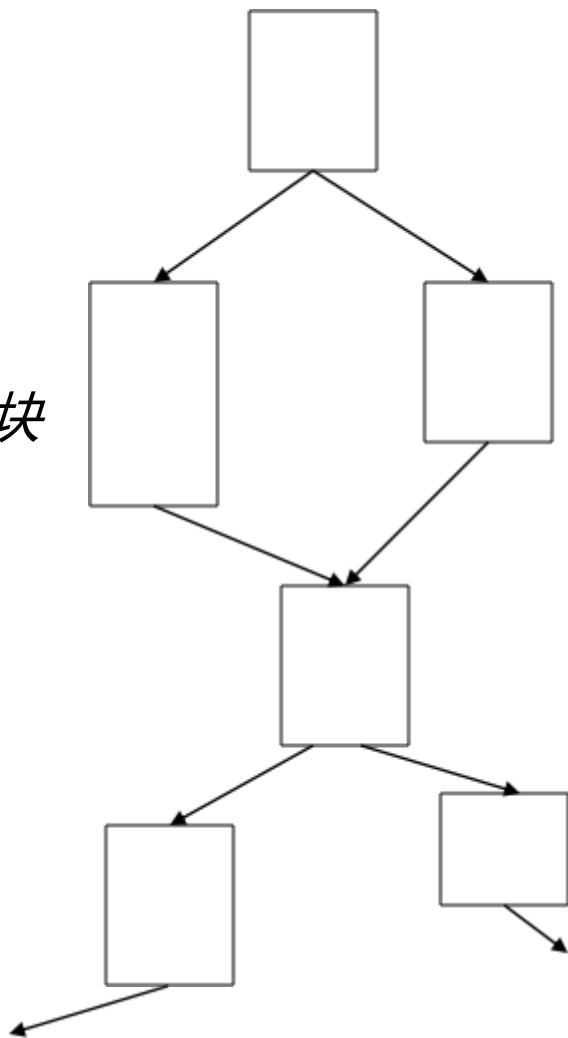
软流水 vs. 循环展开



软流水的启动开销和结束开销整个循环仅一次，而不是每次迭代有一次

如果没有循环语句呢?

基本代码块



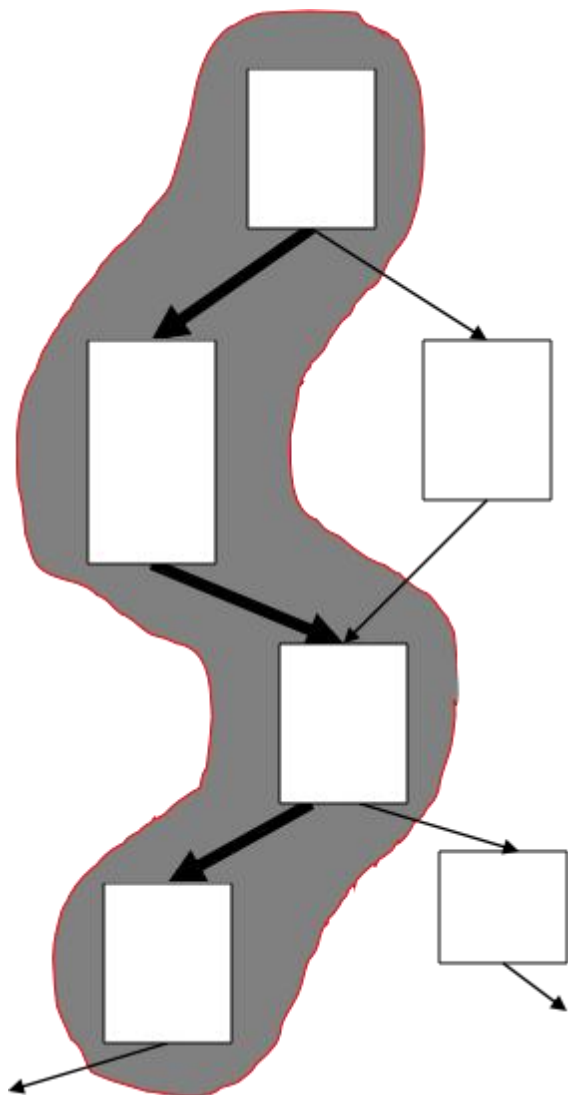
- 分支语句限制了基本代码块(无分支)的大小
- 在每个基本代码块中很难进行指令（操作）并行

基于执行轨迹的调度(trace scheduling)

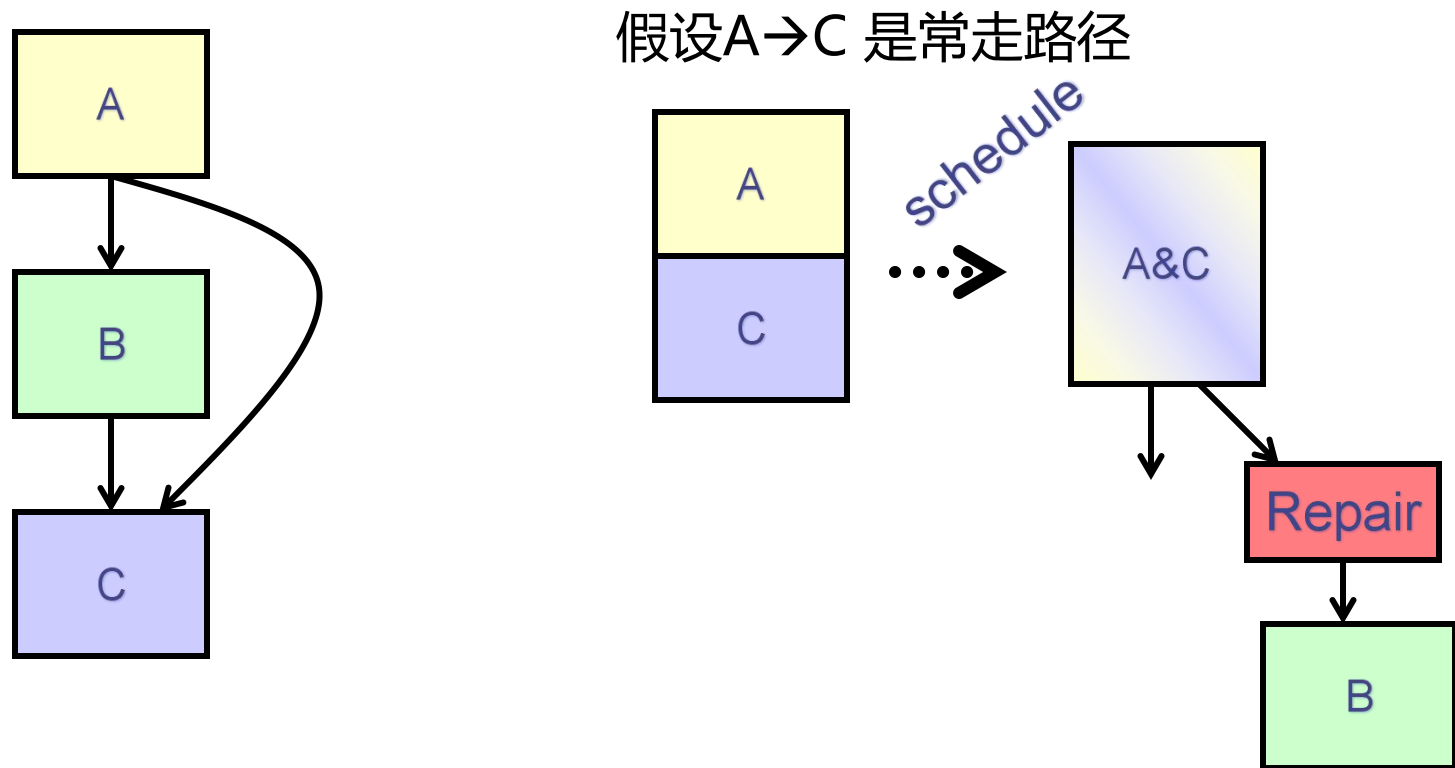
- 分析程序执行轨迹 (最经常走的分支路径)
- 执行轨迹上的代码统一调度
- 增加修复代码，处理不按轨迹执行的情况

如何分析轨迹?

离线profiling，或者通过编译器进行分析



基于执行轨迹的调度(trace scheduling)



- A和C之间有B依赖关系，比较复杂，不太容易高度并行化
- 将A和C合并，这样可以生成一个较大的更容易并行化的代码块，但是这样合并之后，程序逻辑出现问题，通过增加repair代码来修复，使得结果正确

Trace Scheduling Example

test = a[i] + 20;

If (test > 0) then

sum = sum + 10

else

sum = sum + c[i]

c[x] = c[y] + 10

有分支，不易并行化

代码块合并后

test = a[i] + 20

sum = sum + 10

c[x] = c[y] + 10

if (test <= 0) then goto repair

...

修复代码

repair:

sum = sum - 10

sum = sum + c[i]

VLIW存在的问题

□ 分析分支路径困难

- 离线分析开销较大

□ 无法分析的分支语句调度困难

- 最优调度算法与分支路径息息相关

□ 增加内存空间

- 代码填充可能浪费内存/cache空间（可能填不满）
- 循环展开/软流水会增加代码长度

□ 调度内存操作困难

- Caches或者内存bank冲突会引发不确定性
- 内存访问地址未知限制了指令调度

超标量 vs. 超长指令字

Superscalar	VLIW
在一个时钟周期内发射多条指令，多个执行单元并行处理这些指令	将多条指令打包成一个超长指令字
动态调度：硬件在运行时行指令的调度和重排，提高指令级并行性。	静态调度：编译器负责指令打包和调度，硬件设计简化。
复杂的硬件设计	复杂的编译器设计
较高的功耗和热量	低功耗和高效性
灵活性高	灵活性差
适合通用计算任务，如服务器、桌面计算机	适合于特定应用场景，如数字信号处理器（DSP）、嵌入式系统
被广泛采用，如Intel Core、AMD Ryzen	用于一些专用处理器，如Intel Itanium、DSP

多线程

(Multithreading)

多发射的局限性

□ 指令级并行固有的局限性

- 每5条指令1个分支: 如何填满5-way VLIW?
- 需要 (流水线深度 $\times$ 功能单元数) 个无关指令来填满整个流水线
- 硬件设计复杂
 - 容易: 更多指令带宽, 复制功能单元
 - 困难: 增加寄存器和内存端口

□ 提升性能的方法通常也会增加功耗

- 性能提升和功耗增长并不是线性关系(小于线性)

如何进一步提高并行度？

□ 关键是寻找不相关的指令

□ 前面一直关注同一个线程的指令

- 为何不考虑多线程/进程的指令并行？
- 来自不同线程/进程的指令完全不相关

□ 程序员需要参与进来

- 并行编程 ...

使用多线程提升并行性

□ 一种思路: 允许来自不同线程的指令在同一个流水线中执行

– 单线程: 通过forwarding解决数据依赖

– 多线程:

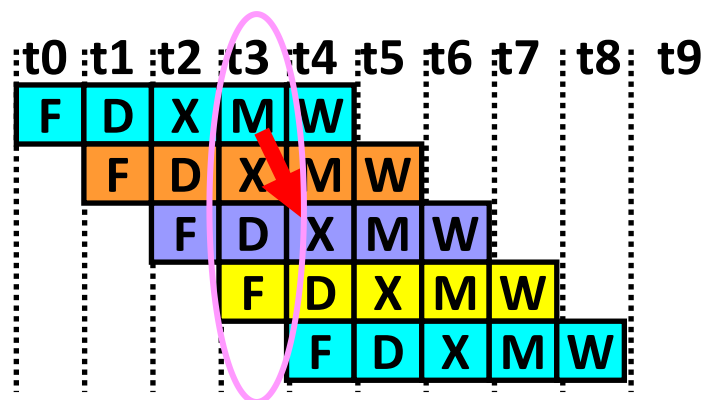
T1: LW r1, 0(r2)

<bubble>

T1: SUB r5, r1, r4

T1: AND r4, r1, r3

T1: SW 0(r7), r5



不需要
forwarding

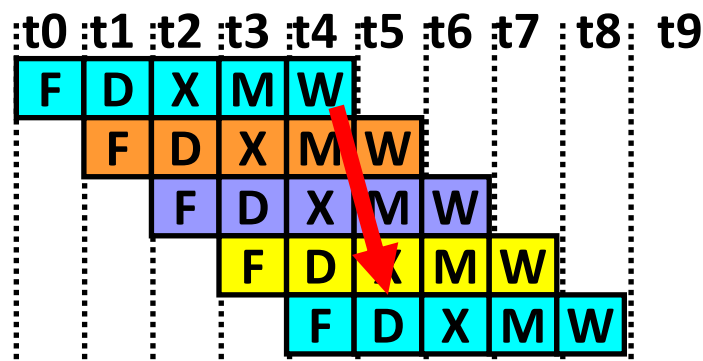
T1: LW r1, 0(r2)

T2: ADD r7, r1, r4

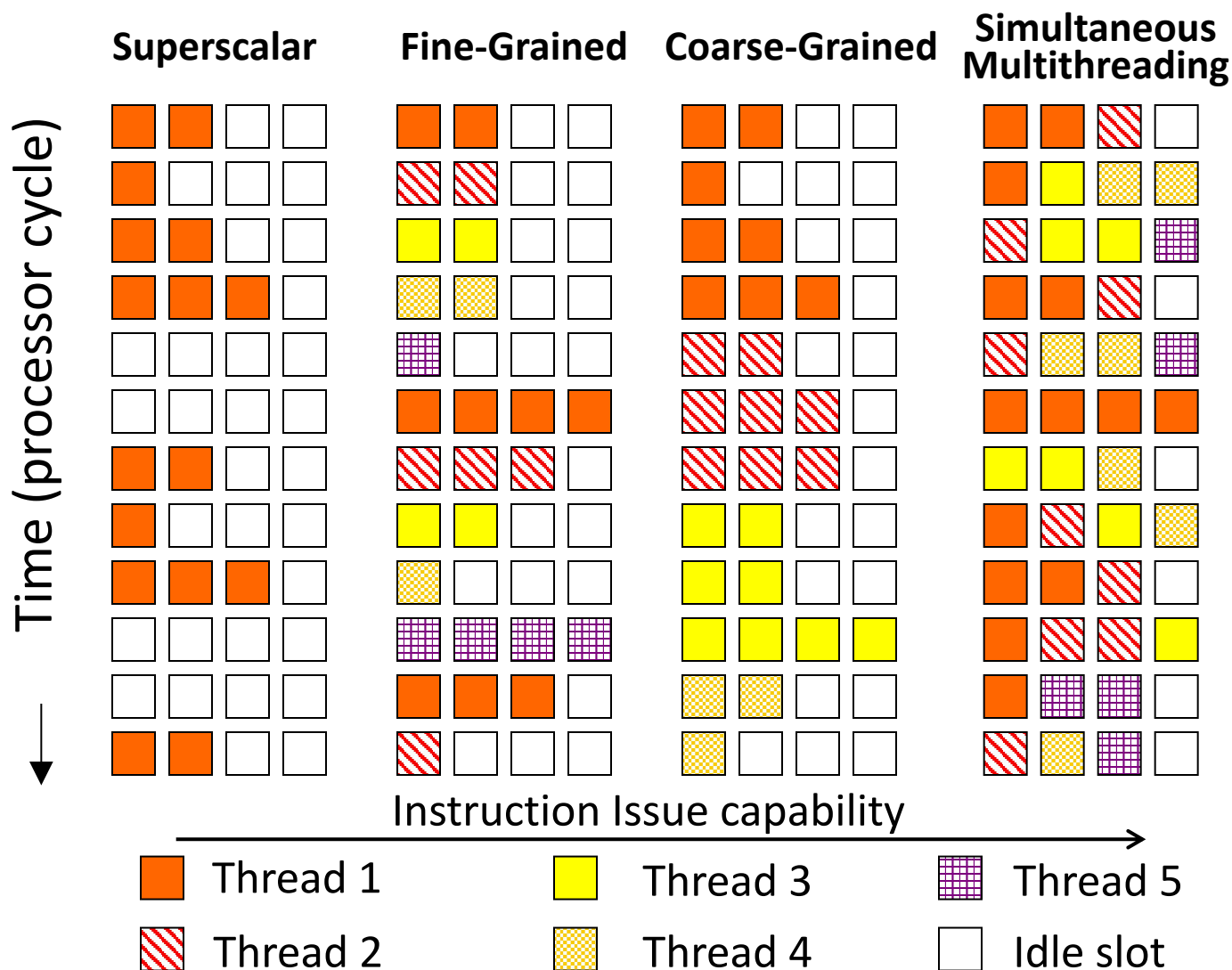
T3: XORI r5, r4, #12

T4: SW 0(r7), r5

T1: SUB r5, r1, r4



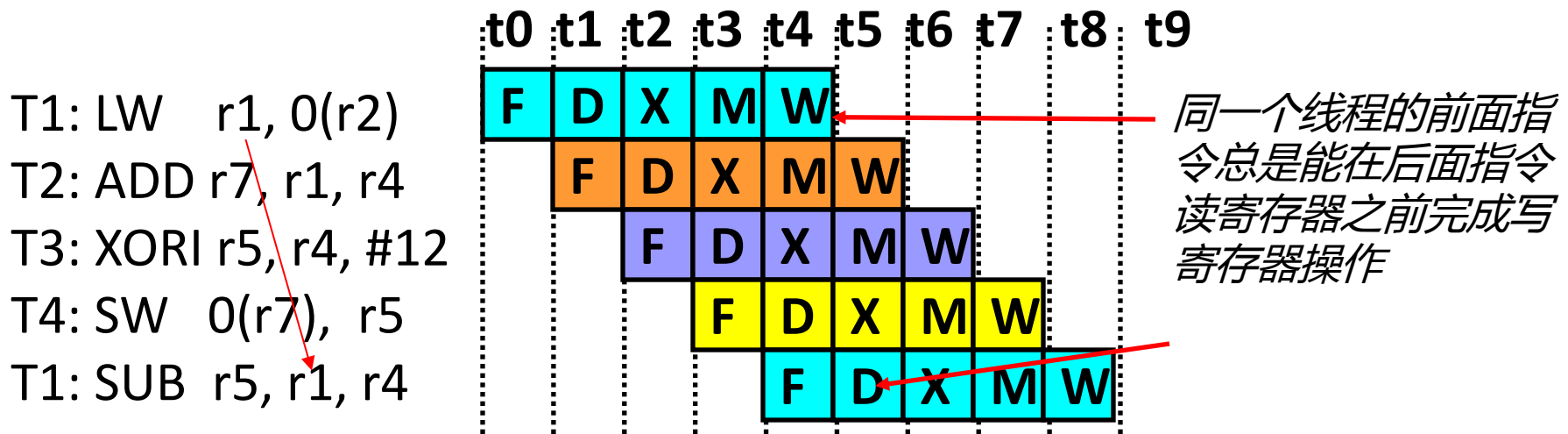
单核多线程执行策略



细粒度多线程(Fine-Grained Multithreading)

□ 在简单流水线上交错执行不同线程的指令

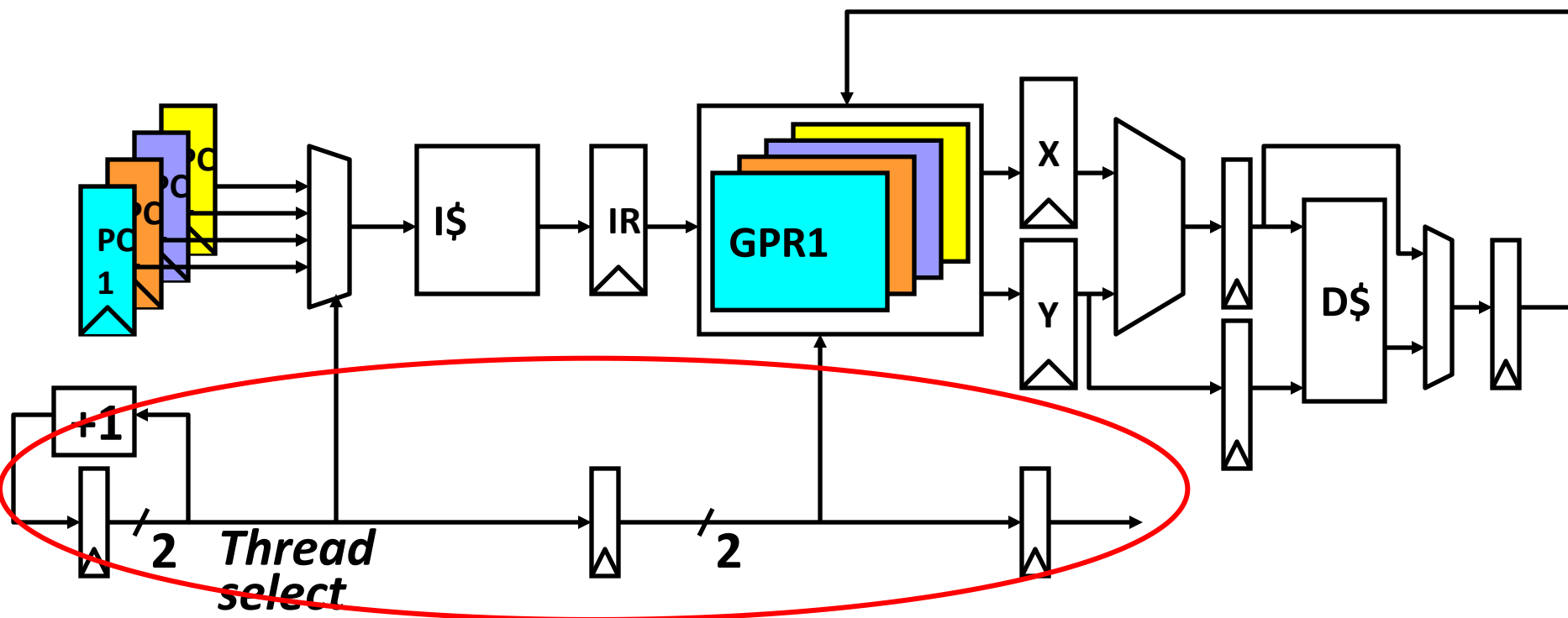
Interleave 4 threads, T1-T4, 5-stage pipe, no internal forwarding



细粒度多线程流水线

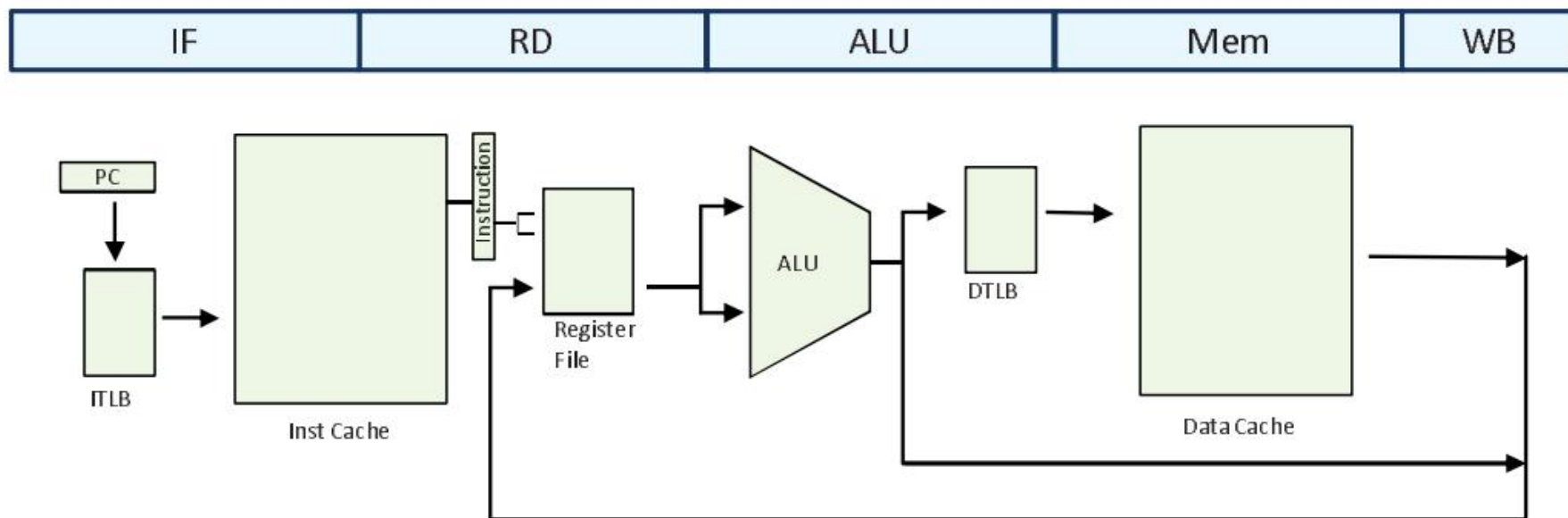
□ 将线程选择信号传递到流水线下游，以确保在每个流水线阶段读取/写入正确的状态位

✓ 在软件看来，好像多个慢速CPU



单线程、顺序、单发射、5级流水线

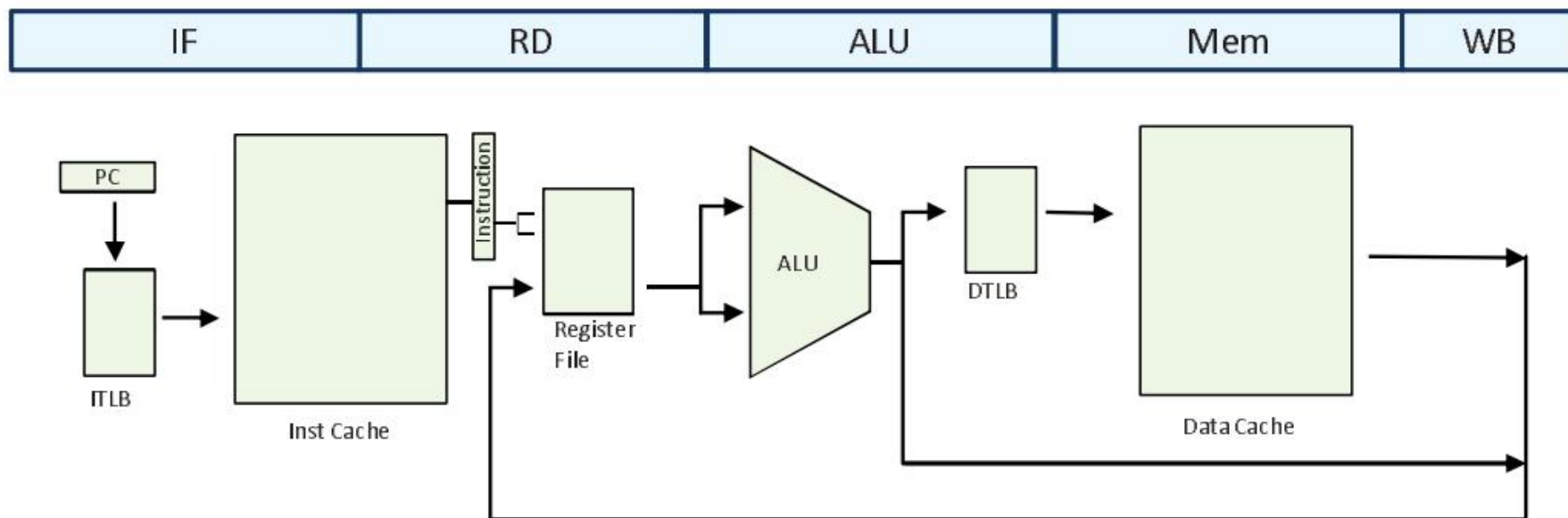
- IF：取指令
 - 先通过ITLB进行地址转换，然后从ICache中取指
- RD：译码 & 读数
- ALU：计算
- MEM：访存
- WB：写回



单线程、顺序、单发射、5级流水线

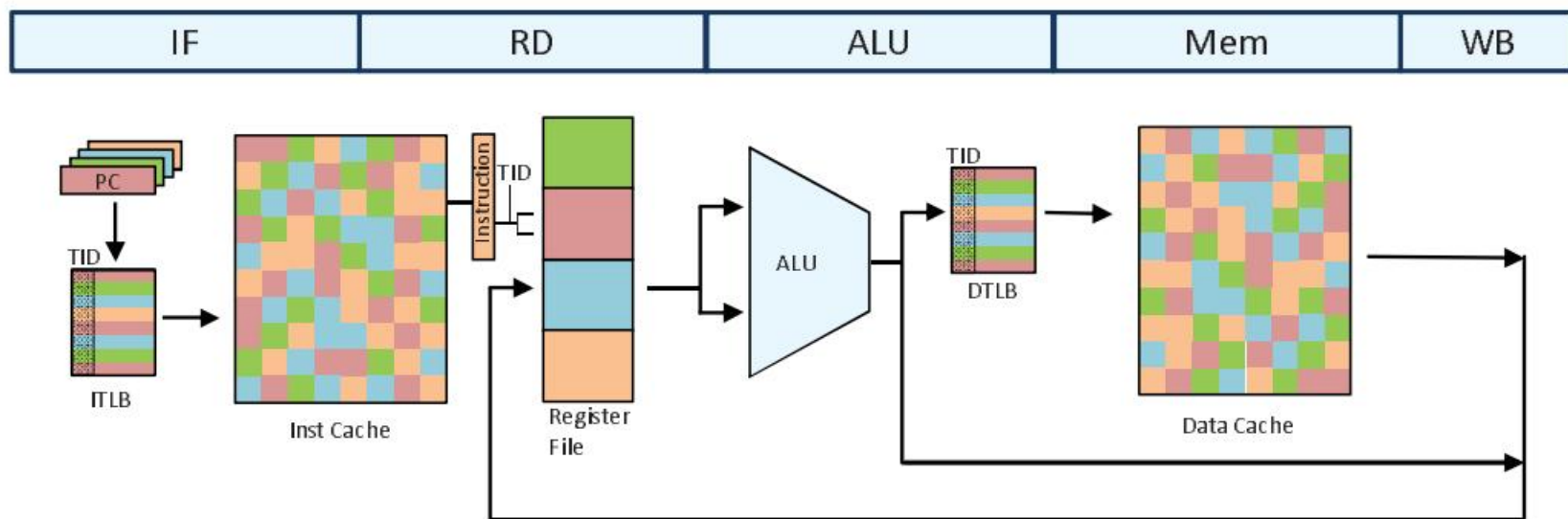
- 原有硬件资源(体系结构状态)

- 32个通用寄存器(GPR)
- 32个CP0寄存器(用于异常处理、虚拟内存管理、系统控制等任务)
- PC(指令计数器)
- 指令和数据cache(物理寻址)



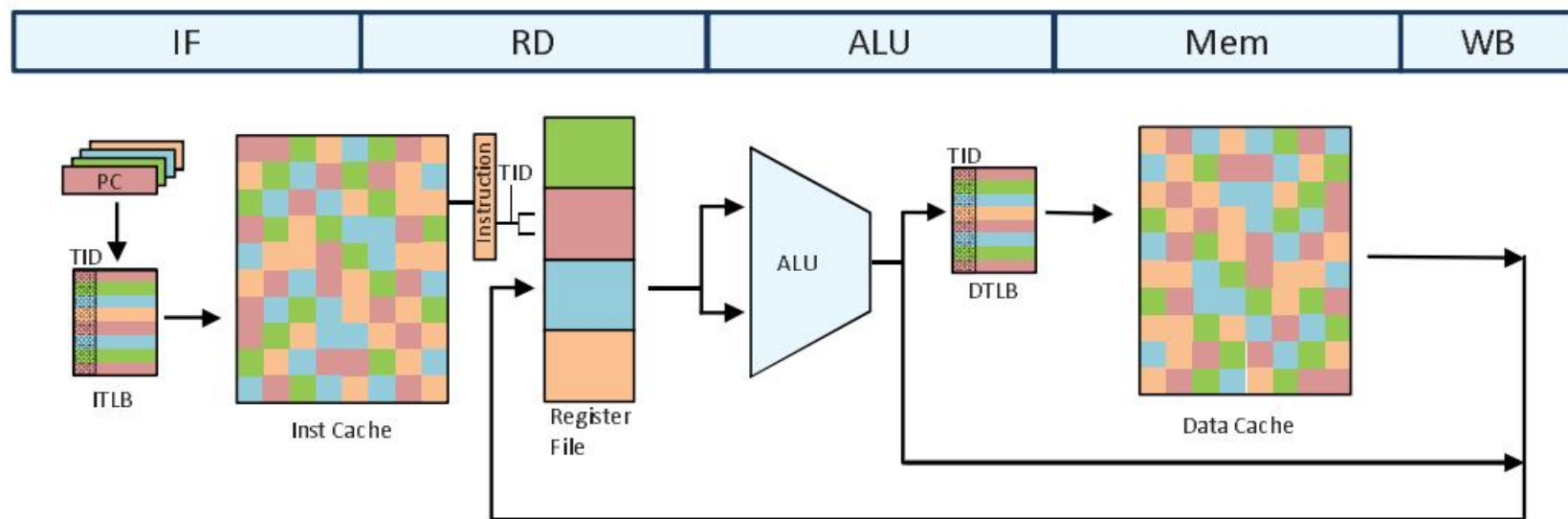
细粒度多线程

- N个hardware context, 支持N线程
- 每个周期都切换线程
- 同一时刻流水线中有多个线程的指令(在不同阶段)同时执行



细粒度多线程

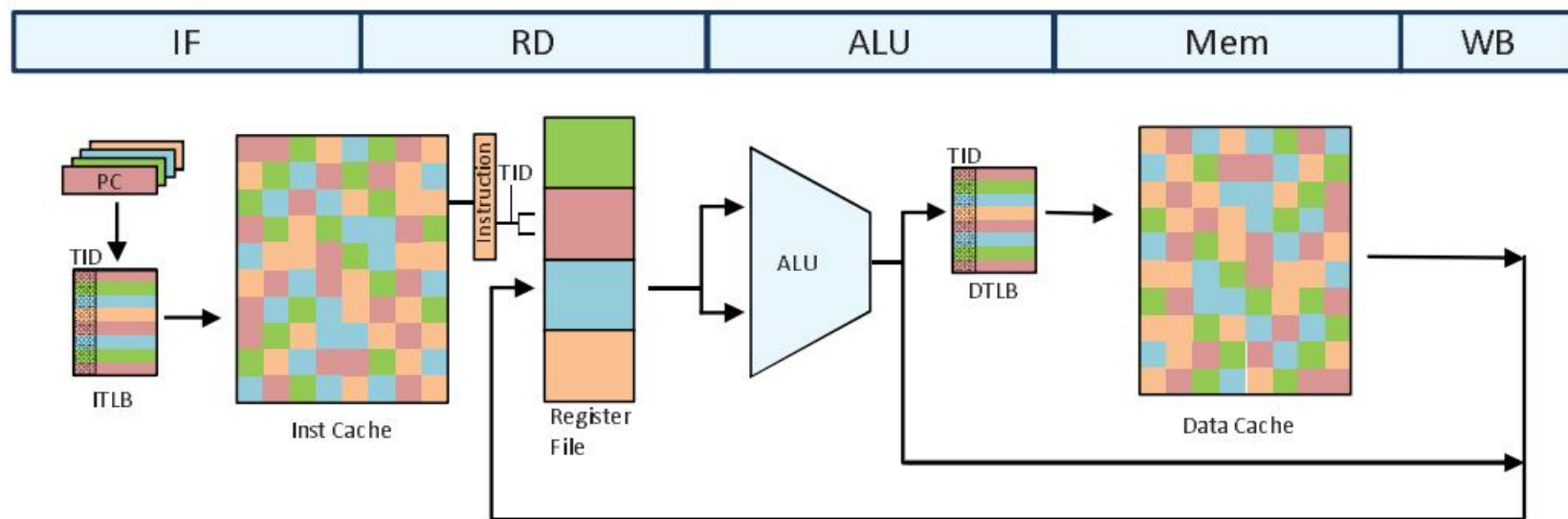
- 硬件资源改动(8个hardware context, 支持8线程)
 - 通用寄存器 (GPR) X 8, 每个线程一份
 - CP0寄存器 X 8, 每个线程一份
 - 1个全局PC + 8个独立PC(每个线程一个)
 - 每条指令增加一个线程标记, 随流水线向后传递
 - 增加全局标记位, 标记每个context是否有线程、每个context中的线程是否在等访存结果等



细粒度多线程

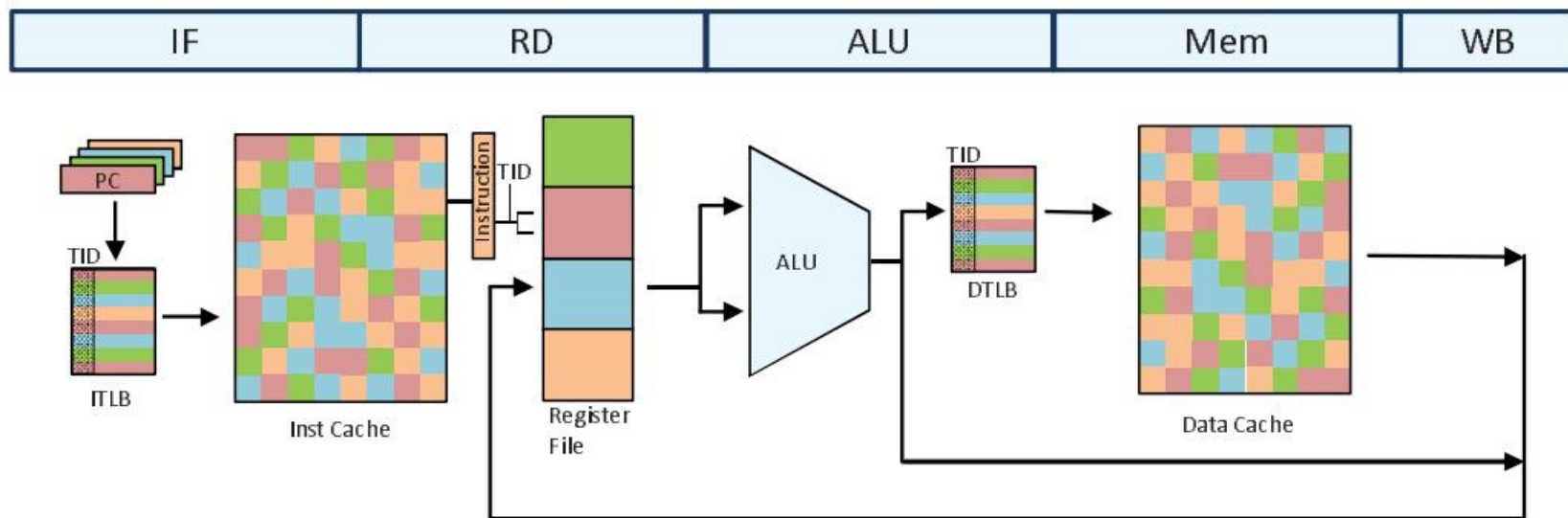
• IF阶段改动

- 1个全局PC(用于取指) + 8个独立PC (每个线程一个)
- 循环方式取指，每个周期切换一次
 - 如果某个线程阻塞，跳过该线程
- ITLB中每个entry增加threadID
 - 防止两个线程虚拟地址相同，无法区分



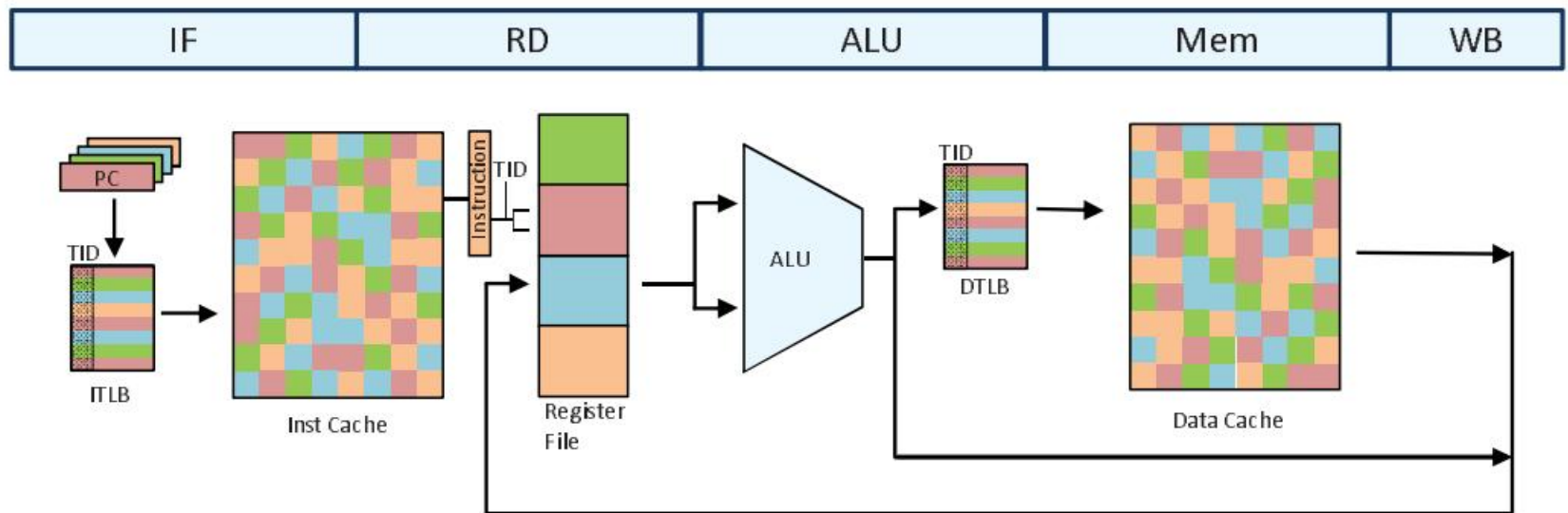
细粒度多线程

- RD阶段改动
 - 寄存器 X8, 每个线程单独一份



细粒度多线程

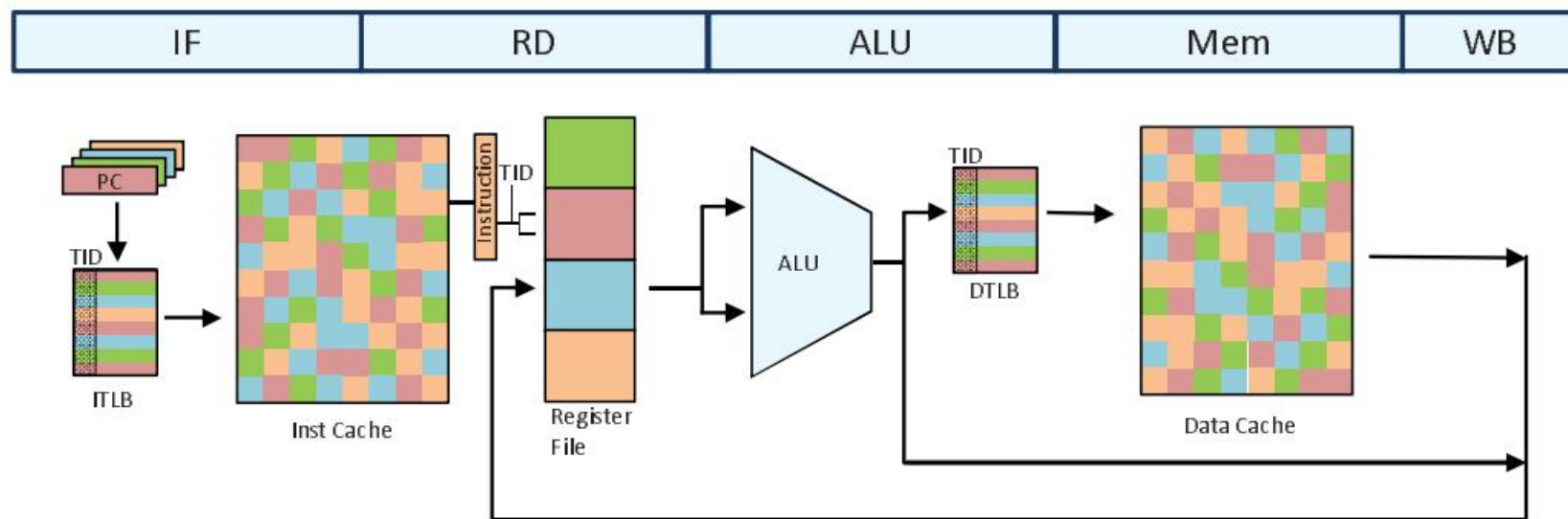
- **ALU阶段改动**
 - 无任何改动



细粒度多线程

MEM阶段改动

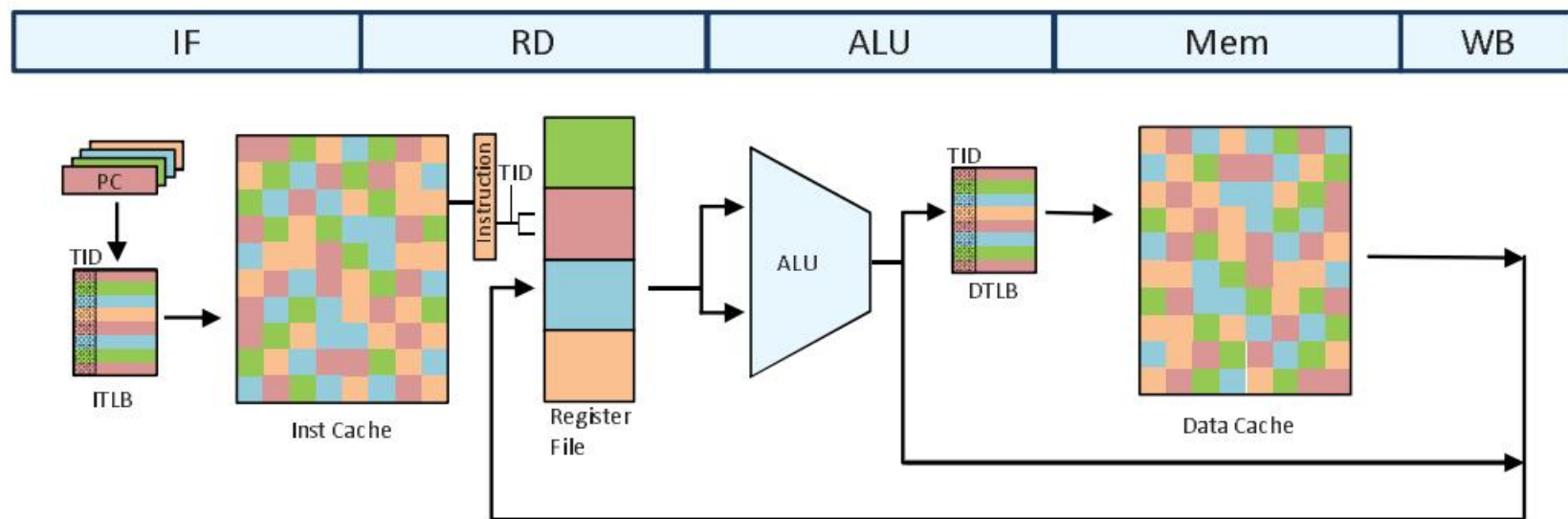
- 访问cache过程和单线程pipeline一样
- 发生cache miss时，将该线程标记为阻塞
 - 该线程的下一条指令将不会进入流水线



细粒度多线程

• WB阶段改动

- 将指令结果写入线程自己的寄存器
- 如果线程曾因为该数据阻塞，将阻塞标记清除



细粒度多线程

□ 优点:

- **能掩盖短时间和长时间流水线停顿**
 - e.g., 内存操作, 有依赖关系的指令, 分支指令等
 - 因为线程切换足够频繁, 每次切换不需要刷新流水线
- **更能充分利用硬件资源**

□ 缺点:

- **单个线程执行时间变长**
 - 一个没有stall的线程可能频繁被切换出去, 被来自其他线程的指令拖慢
- **必须有足够多的线程**
 - 否则无法填满流水线

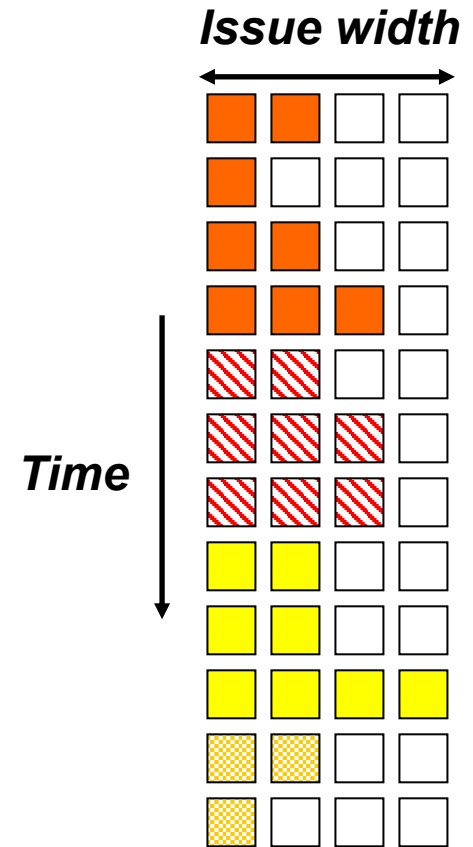
粗粒度多线程(Coarse-Grained Multithreading)

□ 仅当遇到stall时间较长的事件时才切换线程

- 例如cache misses
- 一个线程可以在连续多个周期使用流水线
- 可以掩盖长停顿

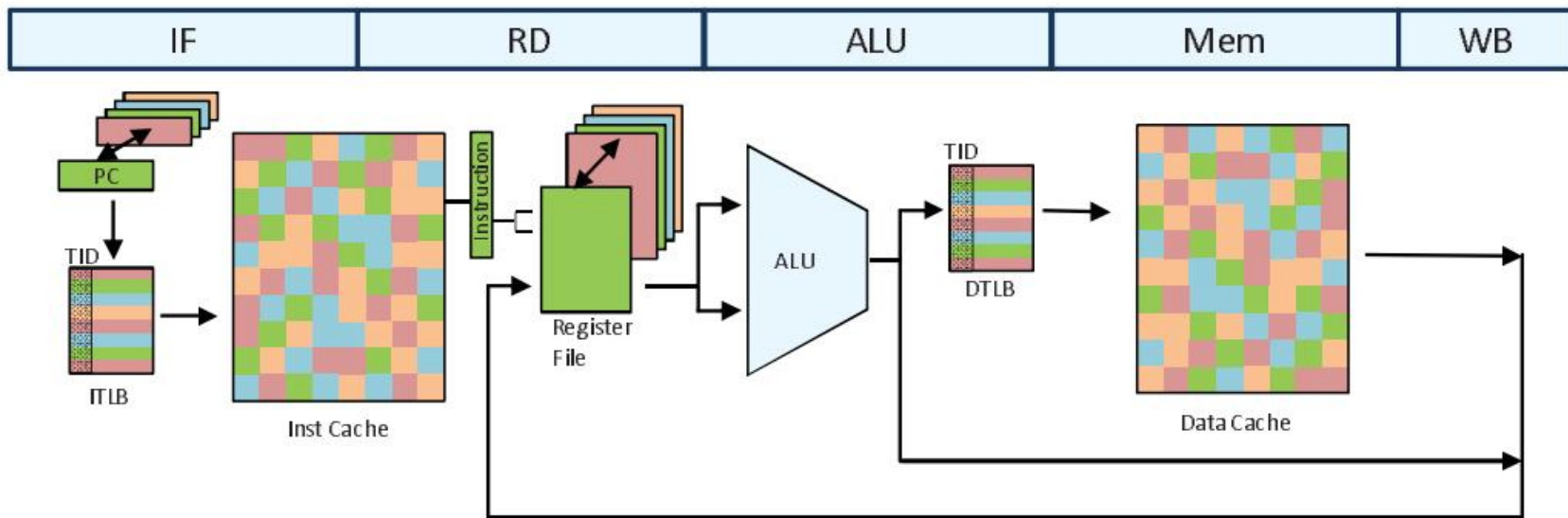
□ 可能的stall事件

- Cache misses
- Synchronization events (e.g., 读取空的队列等待数据而阻塞)
- FP operations



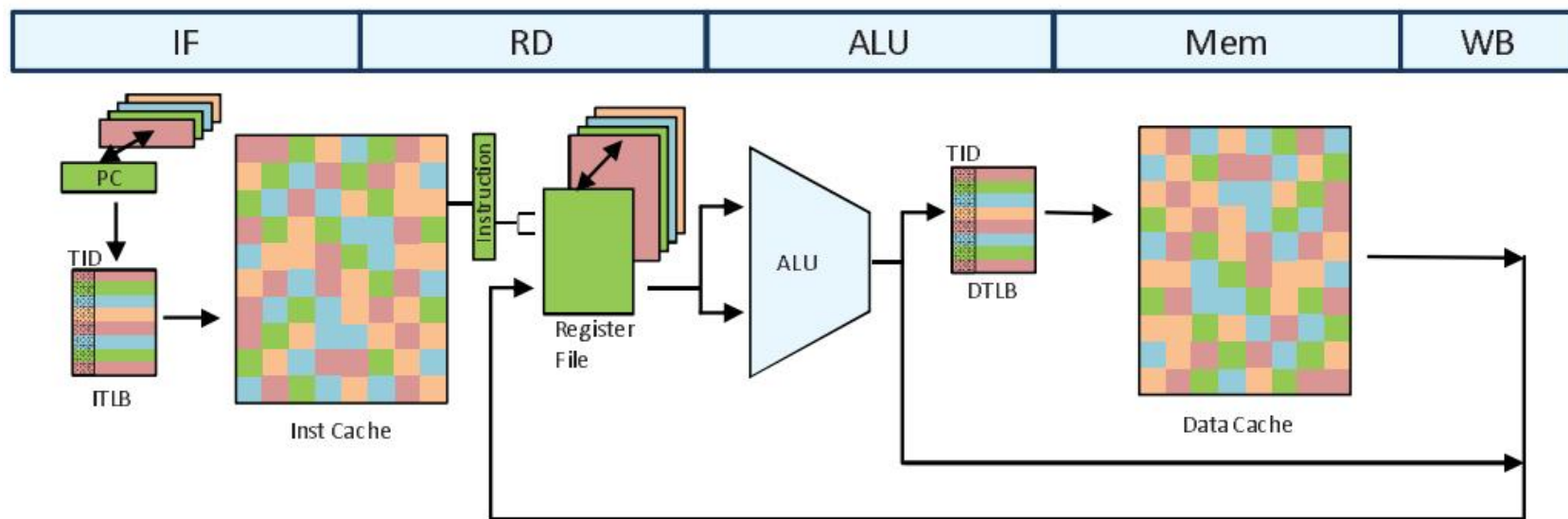
粗粒度多线程

- N个hardware context, 支持N线程
- 同一时间流水线只能被一个线程占用
 - 不允许多个线程的指令共同在流水线中
- 只有发生cache miss时才切换线程
- 发生线程切换时
 - 被换出线程未完成的指令将被清除
 - 换入线程将占用流水线



粗粒度多线程

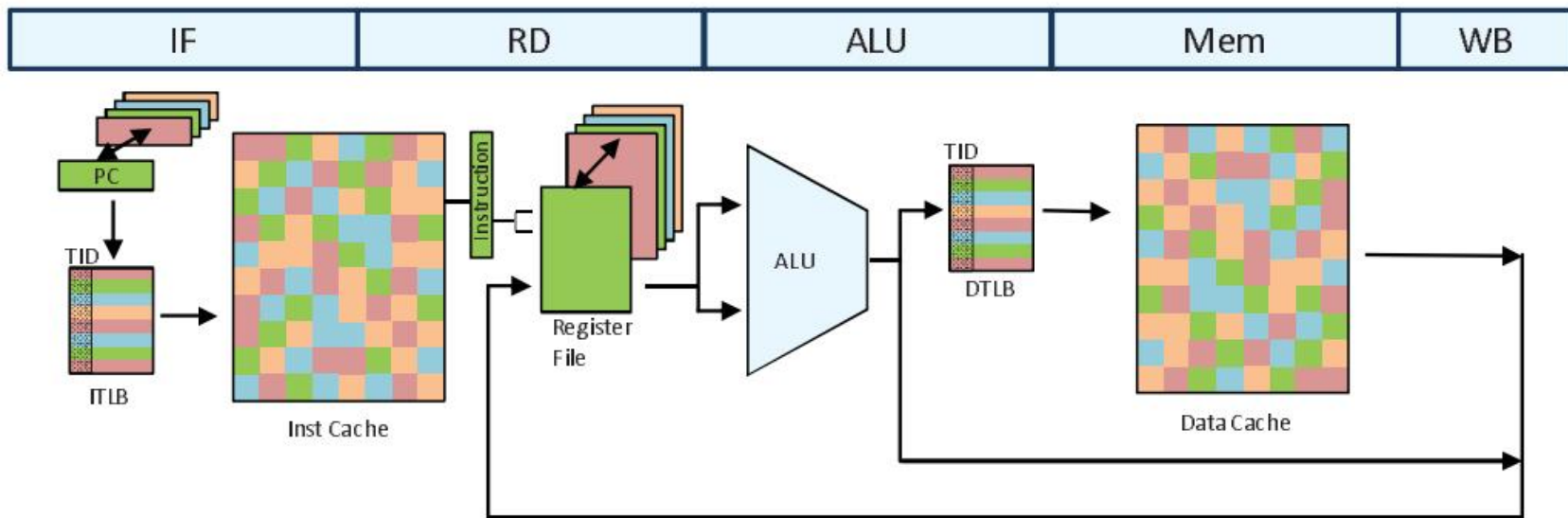
- 硬件资源改动(4个hardware context, 支持4线程)
 - 通用寄存器 (GPR) X 4, 每个线程一份
 - CP0寄存器 X 4, 每个线程一份
 - 1个全局PC + 4个独立PC(每个线程一个)
 - 增加一个全局threadID (2 bits), 标记当前正在运行线程的ID
 - 增加全局标记位, 标记每个context是否有线程、每个context中的线程是否在等访存结果等



粗粒度多线程

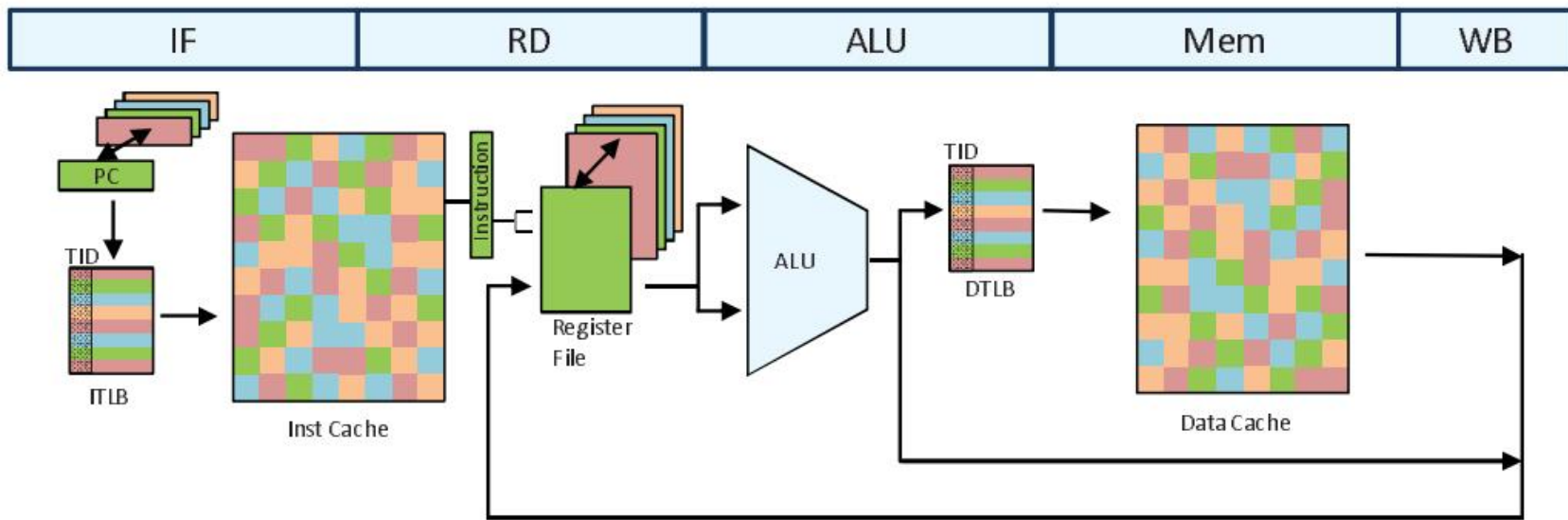
• IF阶段改动

- 1个全局PC(用于取指) + 4个独立PC (每个线程一个)
- ITLB中每个entry增加threadID
- 防止两个线程虚拟地址相同，无法区分
- 发生指令cache miss时，引发context switch
 - 保存当前线程的PC(为发生miss指令的后一条指令地址)
 - 选择另外一个没有阻塞的线程，将全局PC更新为所选线程的PC
 - 刷掉该指令之后进入流水线的指令



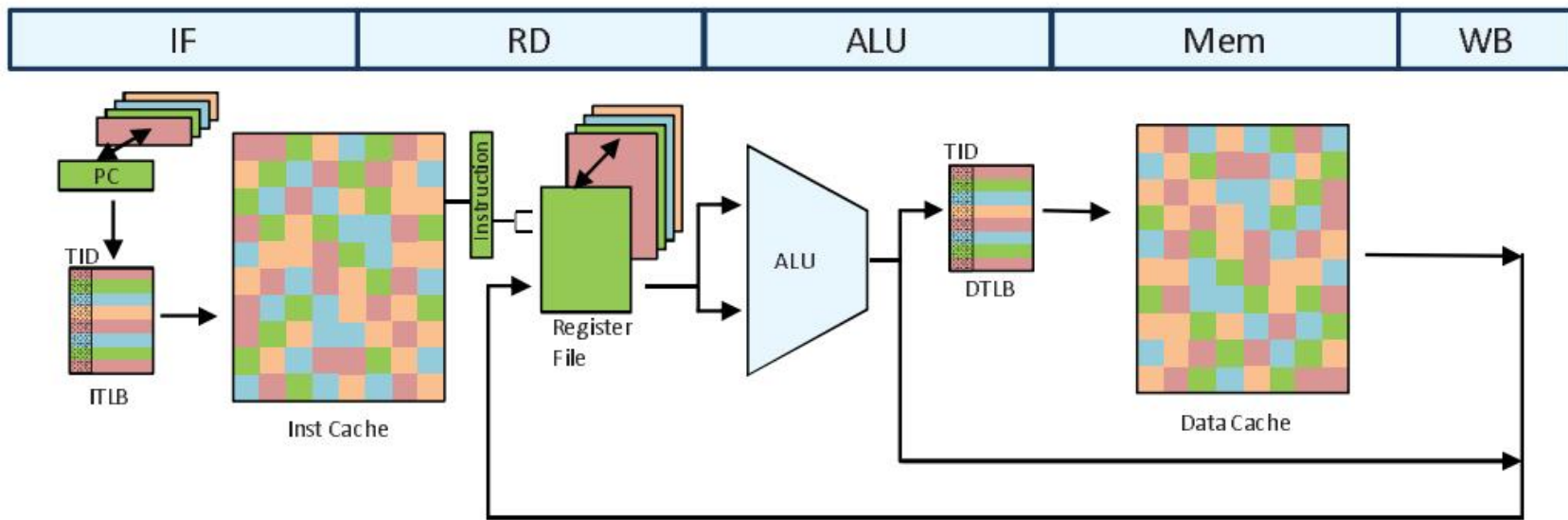
粗粒度多线程

- RD阶段改动
 - 寄存器 X4, 每个线程单独一份



粗粒度多线程

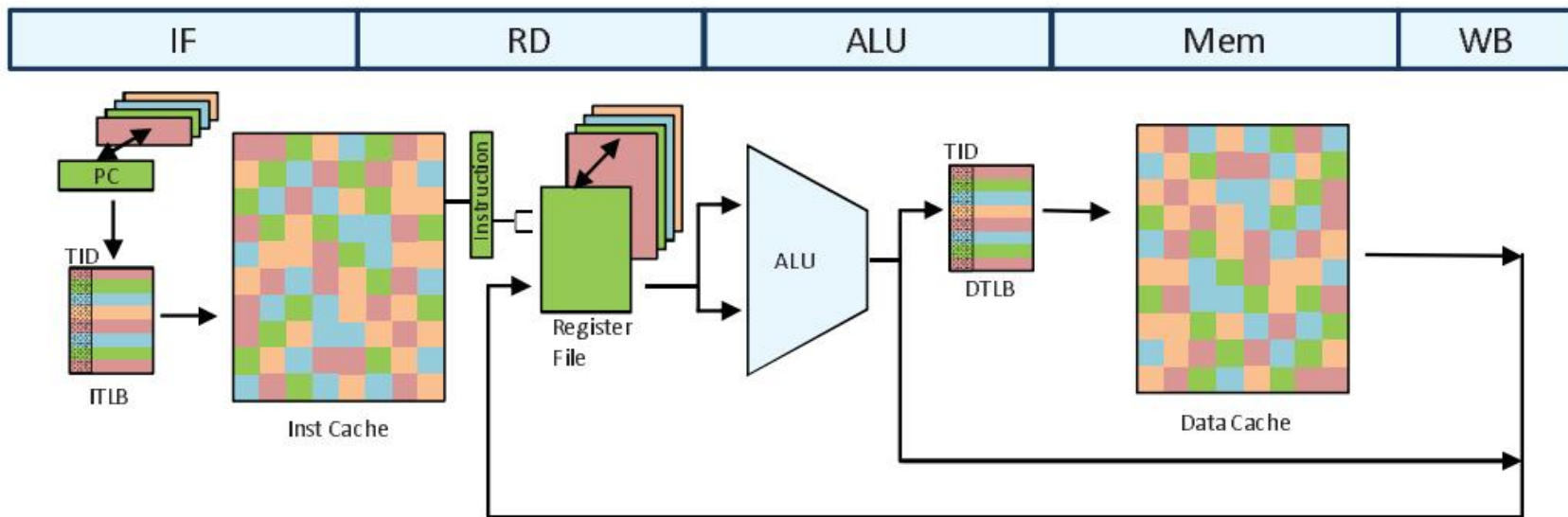
- **ALU阶段改动**
 - 无任何改动



粗粒度多线程

MEM阶段改动

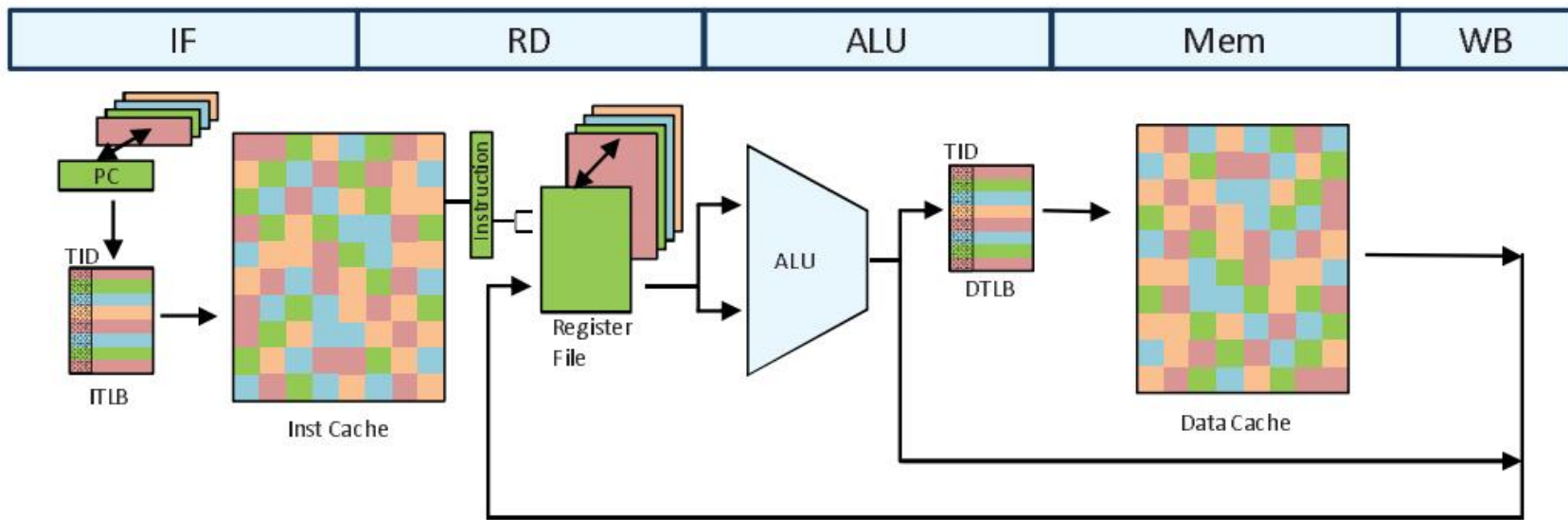
- **cache hit** , 和单线程pipeline一样
- **cache miss时**
 - 将访存指令缓存到load/store buffer
 - 每个线程有单独的load/store buffer
 - 启动context switch , 切换至其他线程
 - 保存当前线程的PC(为发生miss指令的后一条指令地址)
 - 选择另外一个没有阻塞的线程 , 将全局PC更新为所选线程的PC
 - 刷掉该指令之后进入流水线的指令



粗粒度多线程

- **WB阶段改动**

- 将指令结果写入线程自己的寄存器
- 如果线程曾因为该数据阻塞，将阻塞标记清除



粗粒度多线程

□ 优点:

- 线程切换不用那么频繁
- 不会拖慢线程，因为线程只有stall的时候才会被切换出去
- 关键线程需要高优先级

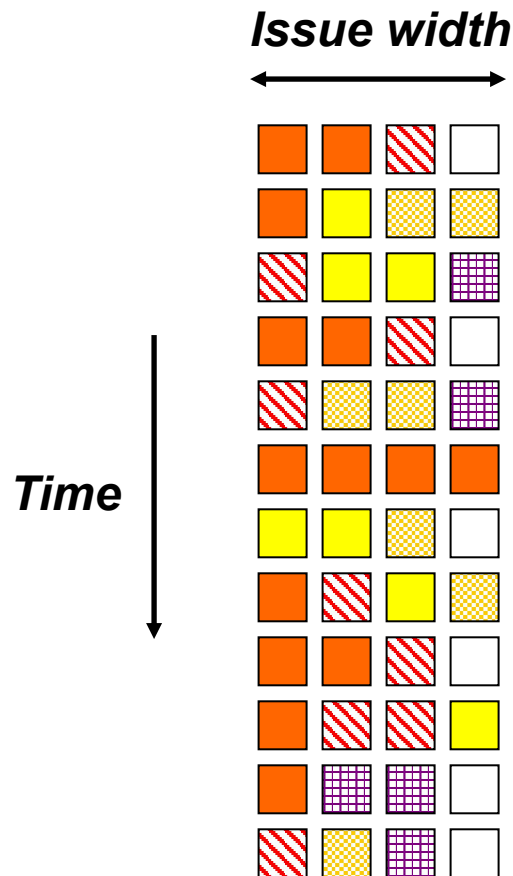
□ 缺点:

- 发生切换时，指令必须排空并重新填充 → 流水线较深时影响较大，适合stall时间很长的情况
- 公平性: stall较少的线程占用流水线时间较长，其他线程可能饿死
 - 解决方案: 占用时间超过一定额度强制切换

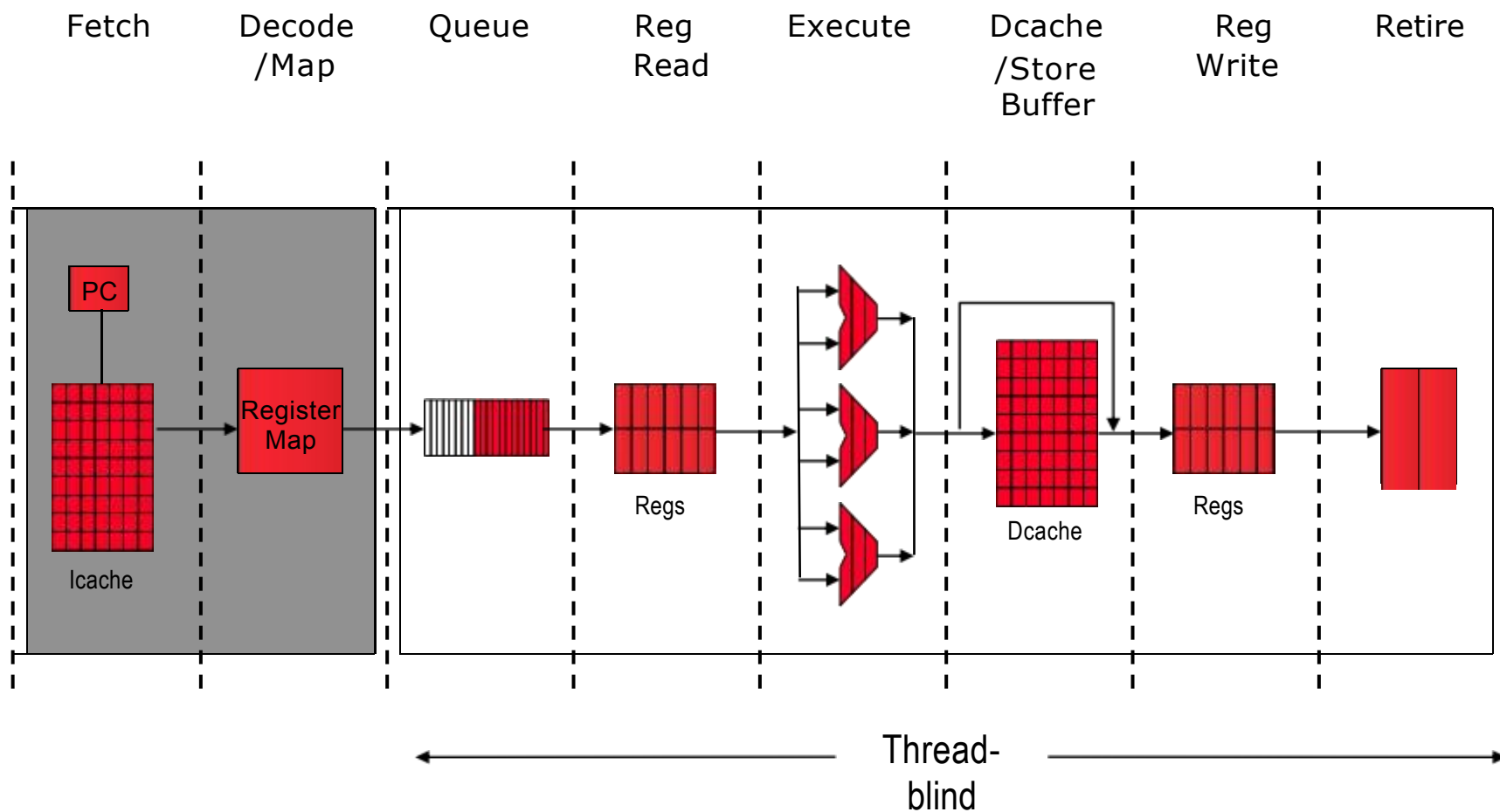
同步多线程(Simultaneous Multithreading,SMT)

□ 允许同一周期发射的多个指令来自不同线程

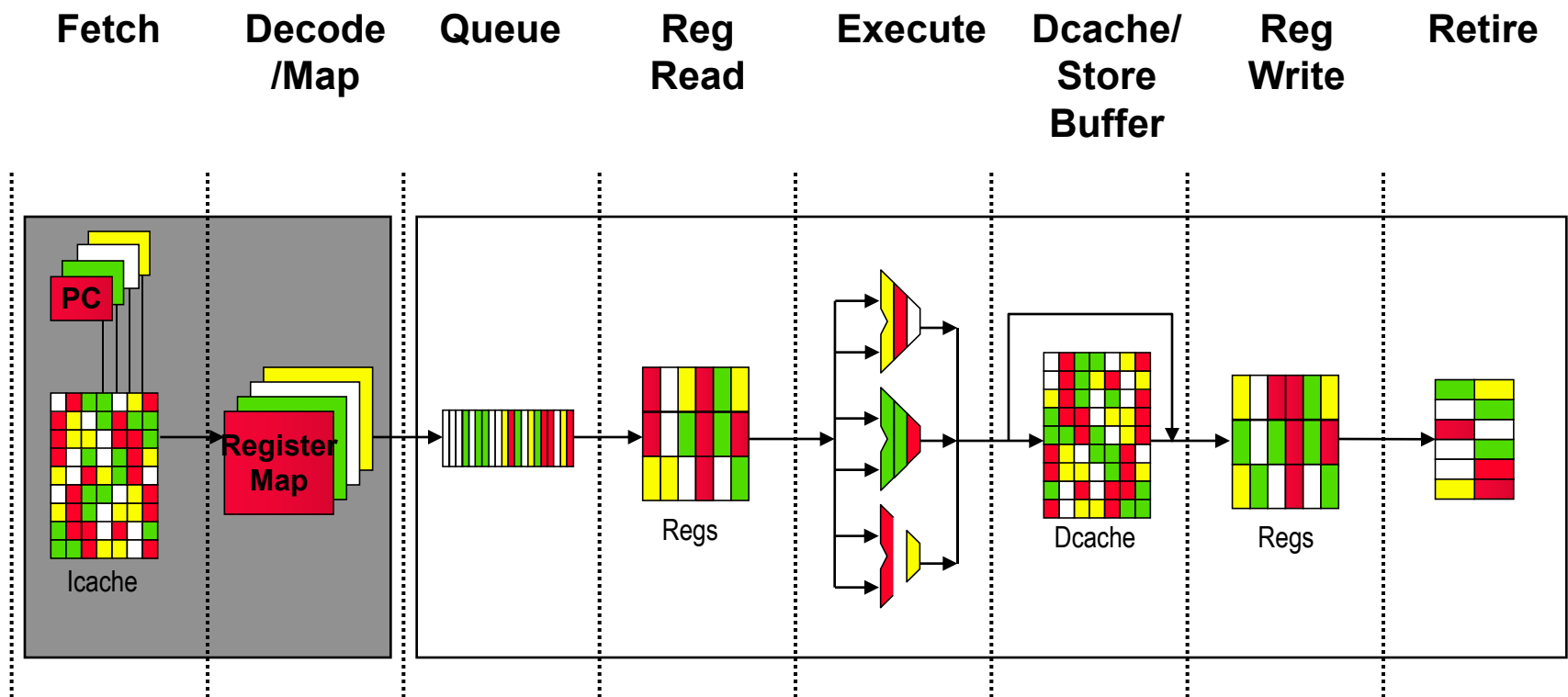
- 保持多个执行单元的高利用率
- 更好的流水线效率
- 多个线程同时在流水线执行，无需上下文切换



单线程乱序执行流水线



SMT乱序执行流水线



SMT 包含4个线程，每个线程外界看来都是顺序执行

SMT的硬件资源

需要复制的资源 (每个线程都有一份)	• return address stack • PC • architected register file • control logic/state
不需要复制，但需要划分的资源 (每个线程分配一部分)	• reorder buffer • load/store buffers • various queues(e.g., scheduling queue)
多个线程共享的资源	• Caches: L1, L2, L3 cache • microarchitecture registers(e.g., physical registers) • execution unit

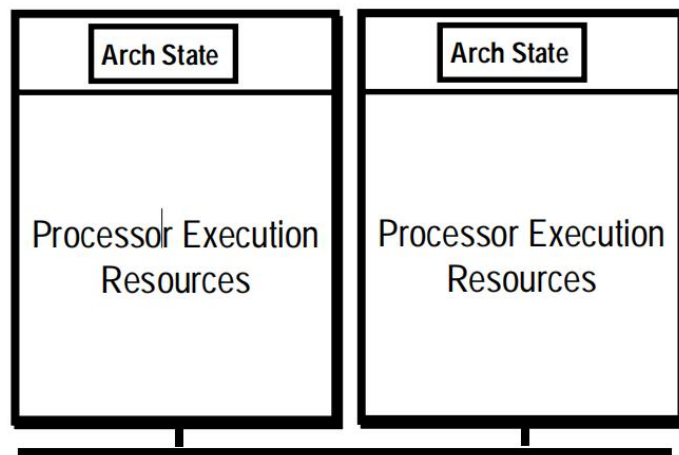
SMT例子：Intel Hyperthreading

- Pentium 4 设计的一部分(Xeon)
- 每个物理核(physical core)支持2个逻辑核(logical core)
- 设计目标
 - 尽量减少实现超线程技术的芯片面积成本
 - 大部分资源共享
 - 额外成本<5%
 - 确保当一个逻辑核被阻塞时，另一个逻辑核能够继续向前推进
 - 逻辑核可能因多种原因而停顿，包括处理缓存未命中、处理分支预测错误，或等待前面指令的结果
 - 通过对缓存队列进行划分或限制使用条目个数，防止两个线程同时执行时，一个线程占用所有资源
 - 确保当只有一个线程时，其性能和无超线程技术相当
 - 单线程允许使用所有处理器资源

SMT 例子：Intel Hyperthreading

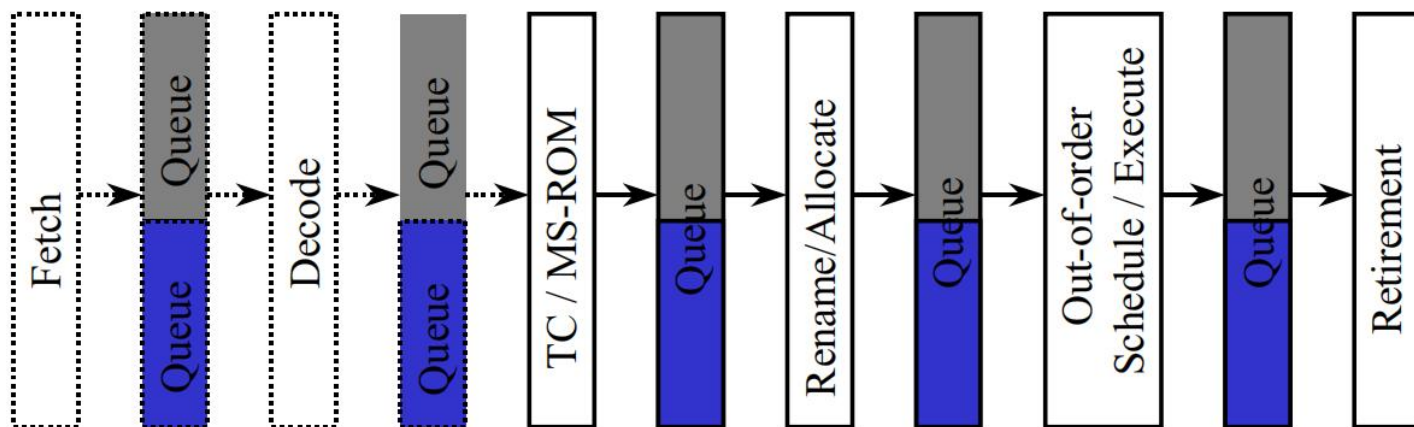
- hyper-threading技术让一个pyshical core看起来像2个logical core
 - 每个logical core有独立的体系结构状态
 - 多个logical core共享一套功能单元
- 操作系统和用户看来，logical core是独立的处理器
- 从微体系结构角度，两个来自logical core的指令将会持续存在并同时在共享的执行资源上执行

Figure 2: Processors without Hyper-Threading Tech



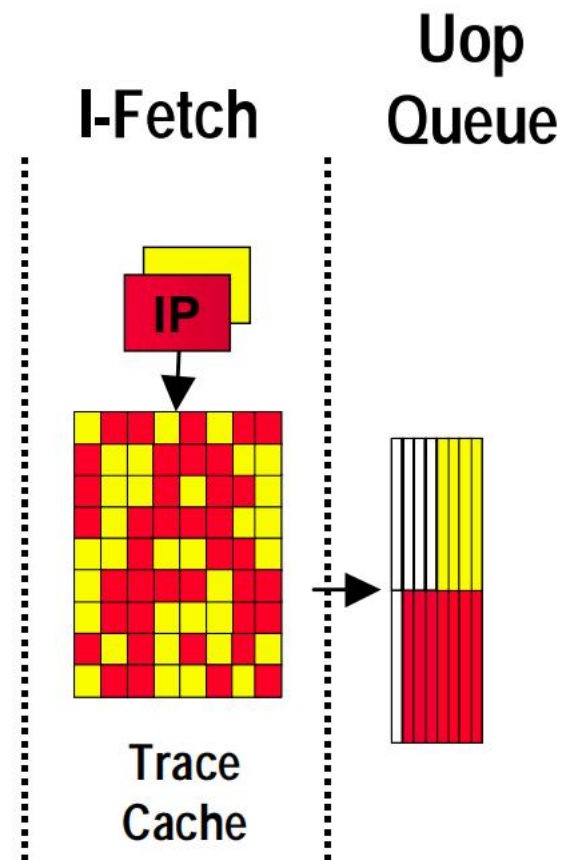
Intel Hyperthreading 总体设计

- 缓存队列将主要的流水线逻辑块分隔开
- 为了确保每个逻辑块的独立向前，缓存队列被划分或复制



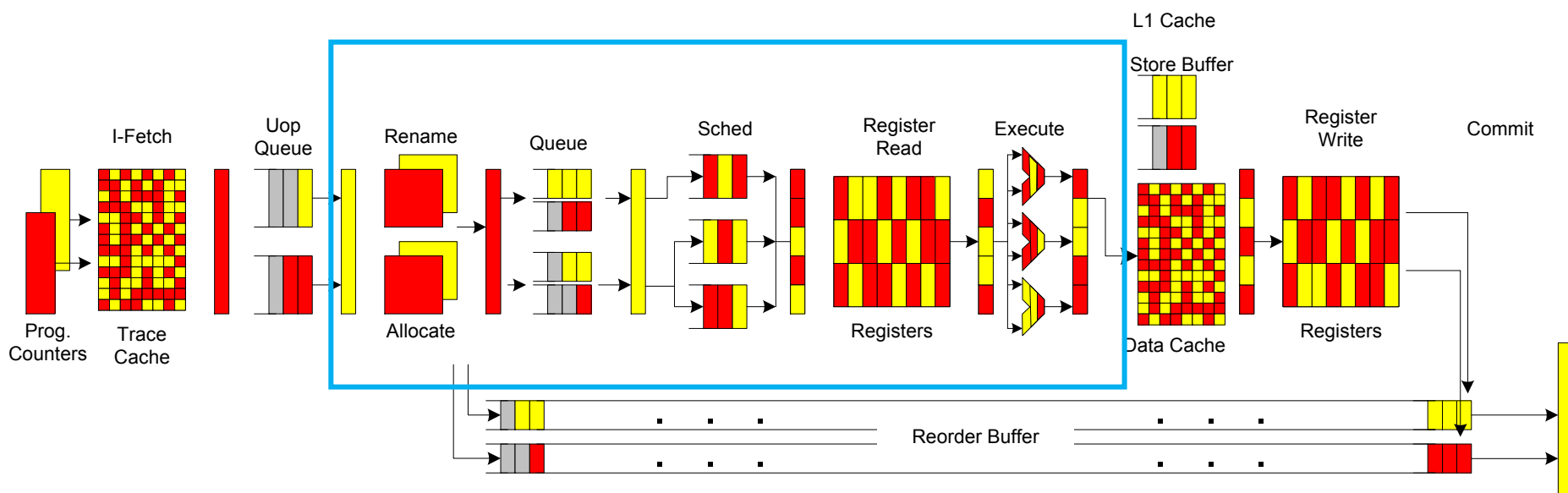
Intel Hyperthreading Front End

- Front end负责为后续阶段提供指令
- 每个logic core维护自己的PC
- 每个周期只有一个logic core能访问I-Cache进行取指
 - 如果都想访问，那么交替进行
- 指令取出后缓存在指令队列，队列划分为两部分，每个logic core使用一部分
- I-Cache由两个logic core共享，每个entry包含线程ID
- 分支预测单元可以共享，也可以单独维护
- return stack buffer每个logic core单独维护



Intel Hyperthreading 乱序执行

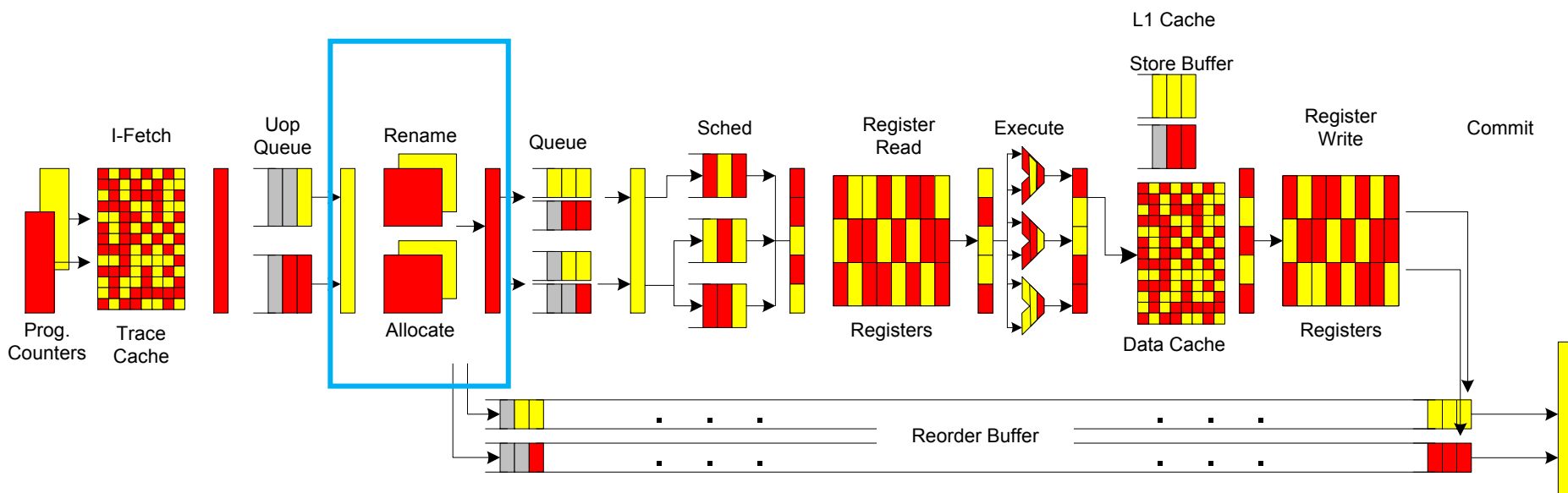
- 乱序执行部分包括分配器、寄存器重命名、指令调度、执行等步骤



Intel Hyperthreading 乱序执行

• 分配器

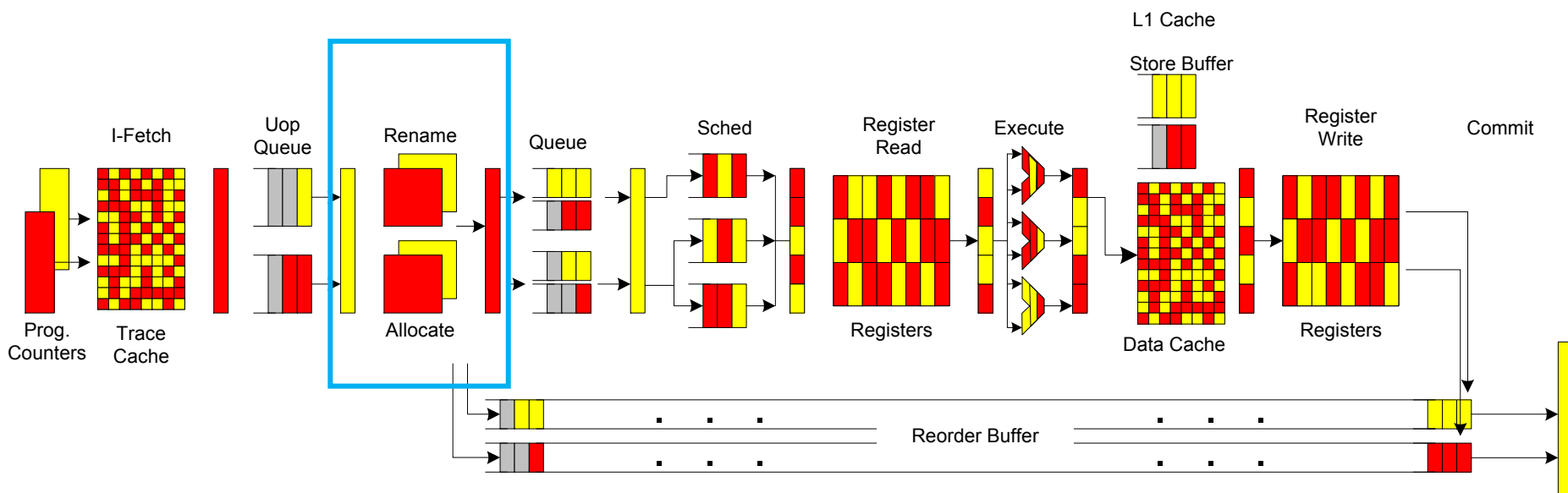
- 用于为指令分配buffers，包括reorder buffer, load/store buffers, 物理寄存器等
- 有些buffer进行了划分，一个线程只能用一半
- 分配器每个周期分配一条指令，两个线程交替着来



Intel Hyperthreading 乱序执行

- **寄存器重命名**

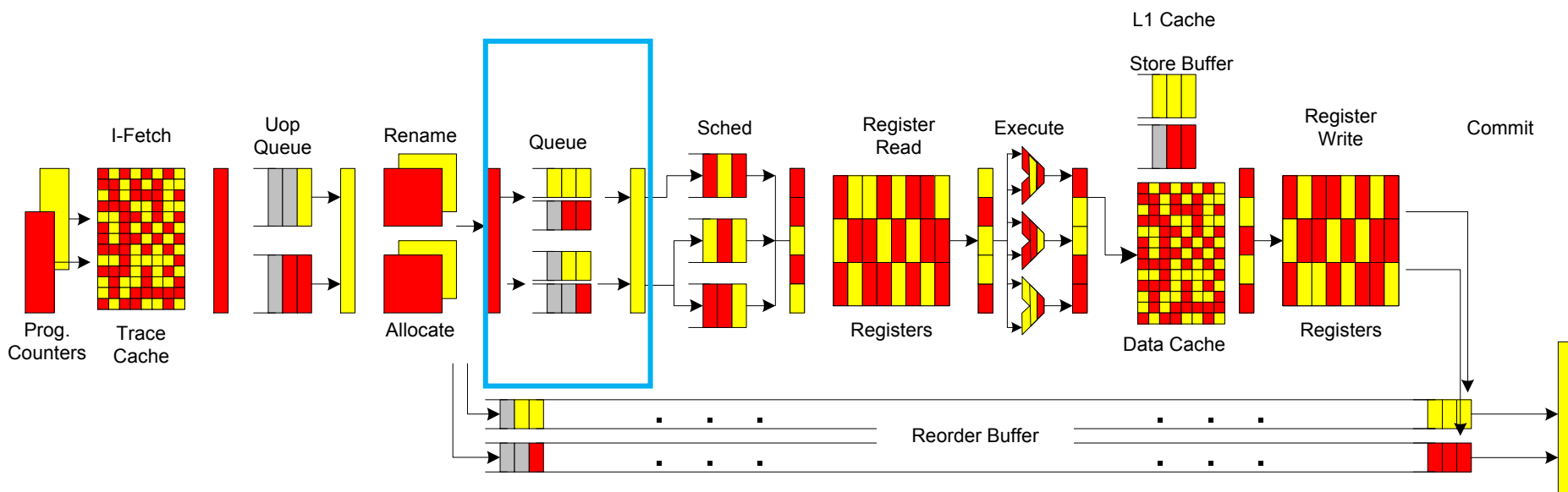
- 将ISA寄存器重命名为物理寄存器
- 所有物理寄存器由两个线程共享
- 分配器每个周期分配一条指令，两个线程交替着来



Intel Hyperthreading 乱序执行

• 指令入队列

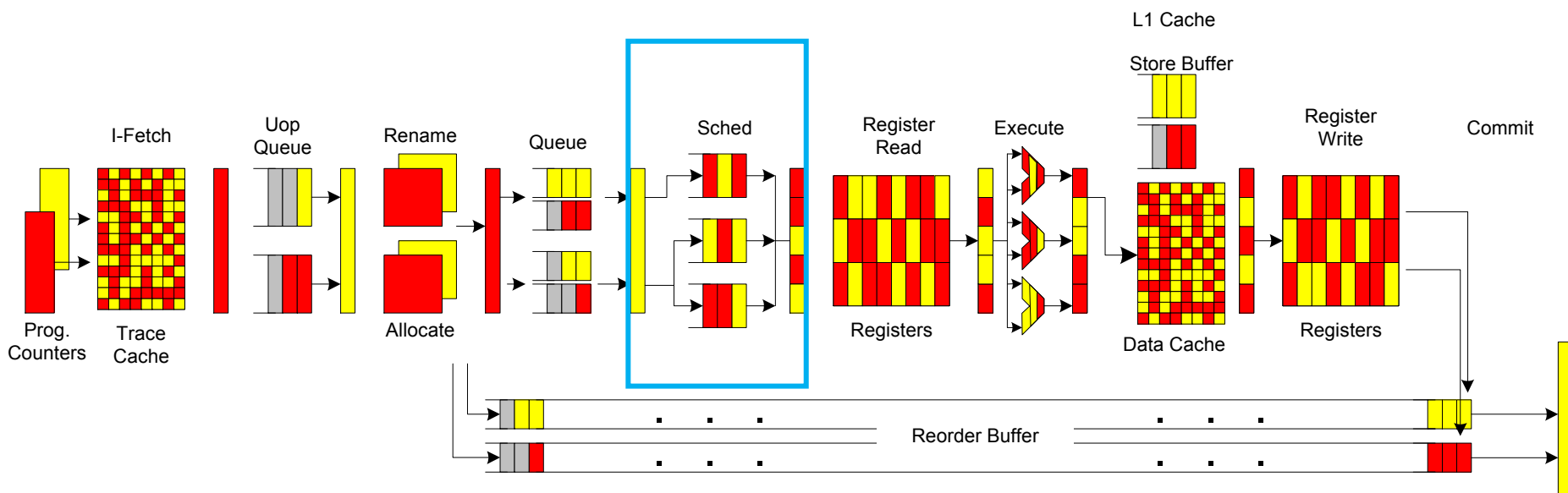
- 寄存器重命名之后，指令将进入对应的指令队列
- 队列有两个，分别缓存内存操作指令和其他指令
- 每个队列都进行了划分，每个线程最多只能用一半



Intel Hyperthreading 乱序执行

• 指令调度

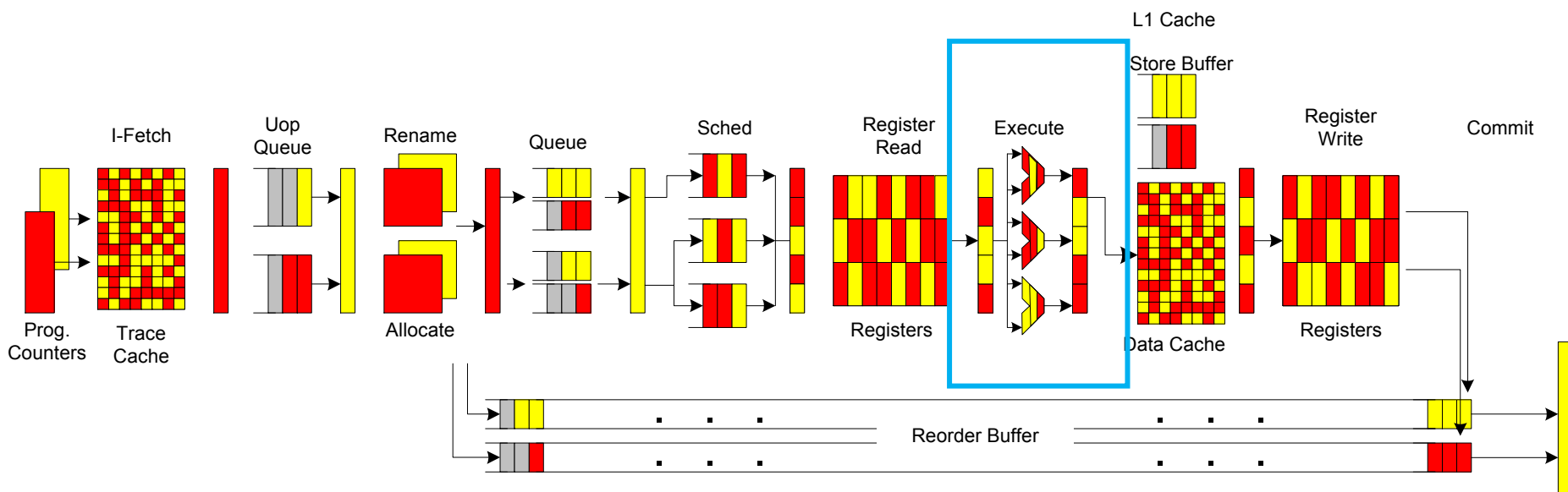
- 每个功能单元都有一个单独的调度器
- 不同调度器可在同一个周期同时调度
- 调度器根据指令操作数状态和执行单元状态决定调度哪些指令送去执行单元
- 每个调度器有一个调度指令队列(8-12个entry)



Intel Hyperthreading 乱序执行

• 执行单元

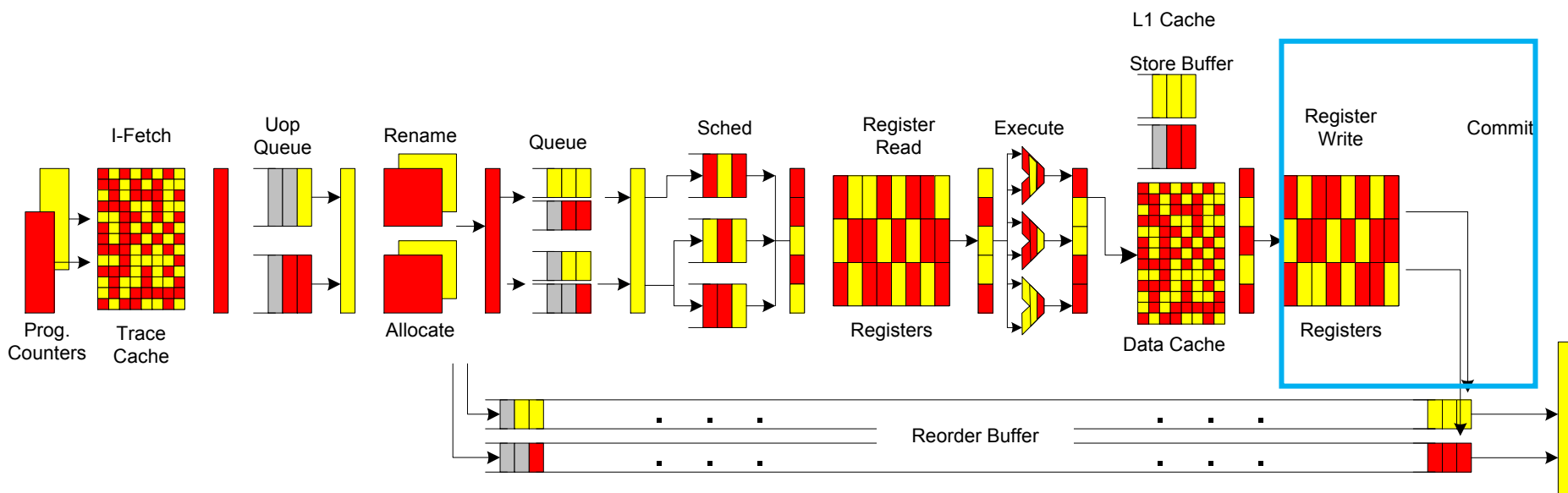
- 执行单元和内存单元都不区分指令来自哪个线程
- 因为寄存器已经重命名为物理寄存器(由两个线程共享)
- 执行结束后，指令进入ROB
- ROB进行了划分，每个线程只能用一半



Intel Hyperthreading 乱序执行

- 提交(retirement)

- ROB中的指令按照顺序提交
- 提交逻辑单元交替提交两个线程的指令
- 如果一个线程无指令提交，那么就提交另一个指令
- store指令提交后，将数据写入L1 cache



Intel Hyperthreading 内存系统

- **包括TLB, L1, L2, L3 cache**
 - TLB由两个线程共享，但每个entry增加线程ID
 - 线程ID用于解决两个线程虚拟地址相同的情况
 - L1 Cache采用写穿透策略，采用virtually addressed + physicall tagged模式
 - L2 , L3 Cache都是采用pysically addressed
- **load和store操作不区分线程，执行过程和单线程一样**